



武漢大學

WUHAN UNIVERSITY

第6讲 决策树





武汉大学
WUHAN UNIVERSITY

01 基本流程

02 划分选择

03 剪枝处理

04 用于回归的决策树

×



武汉大学
WUHAN UNIVERSITY

01 基本流程

Basic workflow of decision tree



- 约翰.罗斯.昆兰 (J. Ross Quinlan) , 悉尼大学教授, 著名人工智能专家, 是一位在机器学习领域具有重要影响力的学者。昆兰于1975年在悉尼大学提出了ID3算法。他的工作特别是**ID3**、**C4.5**和**C5.0**方法的发展为决策树的研究和应用做出了杰出贡献。

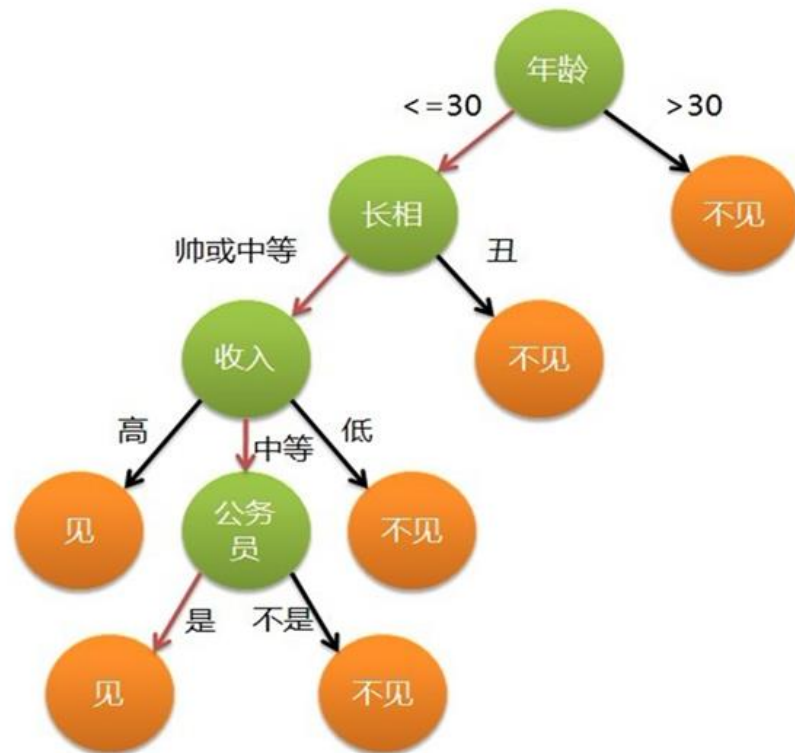


John Ross Quinlan

1 基于**规则**的决策

- 基于规则的思维模式: 人类在进行决策或事物判断时, 一般会基于一些既有规则。制定规则、遵守规则、依据规则并运用规则构成**基于规则的思维**。当规则确定后, 可以通过对一系列问题进行**if-then**式推导而实现最终决策

- 很多时候，规则是**潜在**的。我们只能根据样本的属性取值和既往判断结果来学习规则（有监督的学习）。研究如何从训练数据中**提取**潜在判断规则的机器学习方法，就是“**决策树**（Decision tree）”



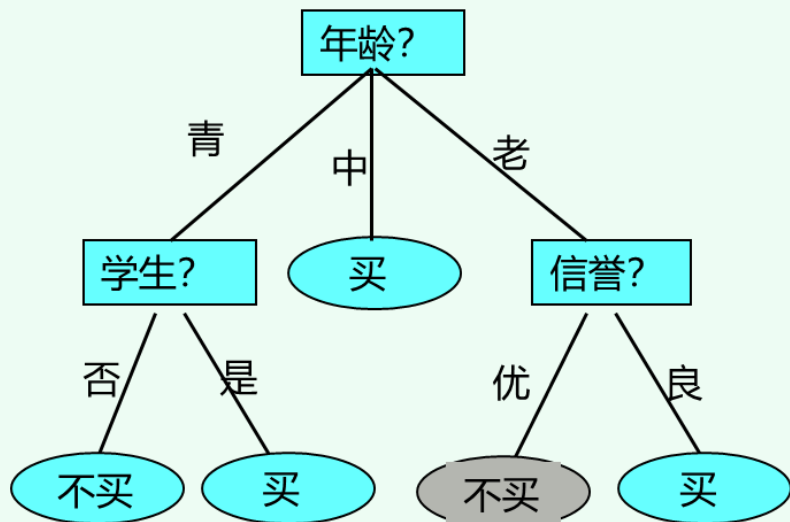
“是否同意相亲”的决策示意图

2 决策树的生成过程

- 决策树中最上面的结点称为**根结点**（root）。每个分支生成一个新的内部结点（internal node），也叫**决策结点**，代表一个问题或者决策。每个**叶结点**（leaf node）代表一种可能的分类结果。**叶节点对应决策结果、其他节点对应属性测试，每个节点包含的样本集合根据属性测试结果被划分到子节点中，根节点包含全体样本。**



根据不同人群购买电脑的数据，构造如下决策树：



计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
128	青	中	否	良	不买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买



- 决策树一般基于**自顶向下**的**贪心算法**（greedy algorithm）构建，常用**递归**来实现。它会在每个结点选择当前状态下**分类效果最好的属性**进行分类，然后继续这个过程，直到这颗树**能够准确地分类训练样本，或者所有的属性都已经被使用过**



- 贪心法是一种在每一步都采取在当前状态下最优的选择，从而希望导致结果是全局最优的算法，是一种非常常见的计算机高级算法。贪心法解决问题的效率很高，它不从整体最优上加以考虑，所做出的仅是在某种意义上的局部最优解。贪心算法中每一步选择都是当前看似最佳的选择，这种选择依赖于已做出的选择，但不依赖于未作出的选择

➤ 给定数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ 和属性集 $A = \{a_1, a_2, \dots, a_d\}$

决策树生成函数记为 $\text{TreeGenerate}(D, A)$:

决策树的生成

```
1: 生成结点 node;  
2: if  $D$  中样本全属于同一类别  $C$  then  
3:   将 node 标记为  $C$  类叶结点; return (A)  
4: end if  
5: if  $A = \emptyset$  OR  $D$  中样本在  $A$  上取值相同 then  
6:   将 node 标记为叶结点, 其类别标记为  $D$  中样本数最多的类; return (B)  
7: end if  
8: 从  $A$  中选择最优划分属性  $a_*$ ;  
9: for  $a_*$  的每一个值  $a_*^v$  do  
10:  为 node 生成一个分支; 令  $D_v$  表示  $D$  中在  $a_*$  上取值为  $a_*^v$  的样本子集;  
11:  如果  $D_v$  为空 then  
12:    将分支结点标记为叶结点, 其类别标记为  $D$  中样本最多的类; return (C)  
13:  else  
14:    以  $\text{TreeGenerate}(D_v, A \setminus \{a_*\})$  为分支结点  
15:  end if  
16: end for
```

输出: 以 node 为根结点的一棵决策树

递归出口

实现方
式之一:
递归实
现、深
度优先





TreeGenerate(D, A) {

1

1. 从A中选出最优划分属性 “年龄”
 2. 依照 “年龄” 的属性值划分了3个子数据集
- }

年龄

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
128	青	中	否	良	不买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买



```
TreeGenerate(D, A) {
```

1. 从A中选出最优划分属性 “年龄”

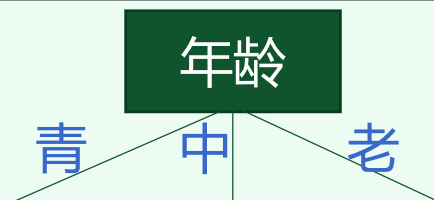
2. 依照 “年龄” 的属性值划分了3个子数据集{

```
TreeGenerate( $D_{\text{年龄1}}$ ,  $A \setminus \{\text{年龄}\}$ ) ; 1.1
```

```
TreeGenerate( $D_{\text{年龄2}}$ ,  $A \setminus \{\text{年龄}\}$ ) ; 1.2
```

```
TreeGenerate( $D_{\text{年龄3}}$ ,  $A \setminus \{\text{年龄}\}$ ) ; 1.3
```

```
}  
}
```


 $D_{\text{年龄1}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	青	中	否	良	不买
64	青	低	是	良	买
64	青	中	是	优	买

 $D_{\text{年龄2}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
128	中	高	否	良	买
64	中	低	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买

 $D_{\text{年龄3}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
132	老	中	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买


 $\text{TreeGenerate}(D_{\text{年龄1}}, A \setminus \{\text{年龄}\})$

{

1. 从 $\{A \setminus \{\text{年龄}\}\}$ 中选出最优划分属性 “学生”

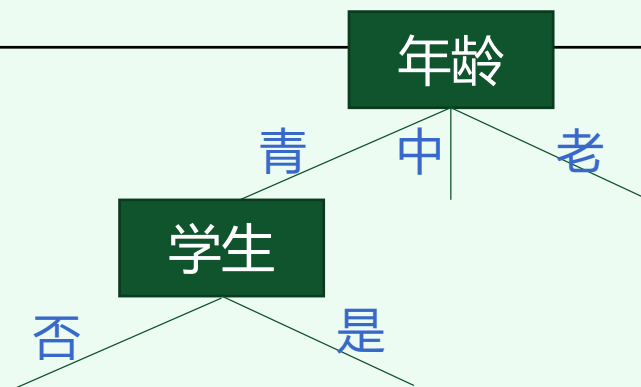
2. 依照 “学生” 的属性值划分了2个子数据集{

 $\text{TreeGenerate}(D_{\text{年龄1-学生1}}, A \setminus \{\text{年龄、学生}\})$; 1.1.1

 $\text{TreeGenerate}(D_{\text{年龄1-学生2}}, A \setminus \{\text{年龄、学生}\})$; 1.1.2

}

}

 $D_{\text{年龄1-学生1}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	青	中	否	良	不买

 $D_{\text{年龄1-学生2}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	低	是	良	买
64	青	中	是	优	买



TreeGenerate($D_{\text{年龄1-学生1}}$, $A \setminus \{\text{年龄, 学生}\}$)

1.1.1

{

1. $D_{\text{年龄1-学生1}}$ 中的样本属于同一类 “不买”

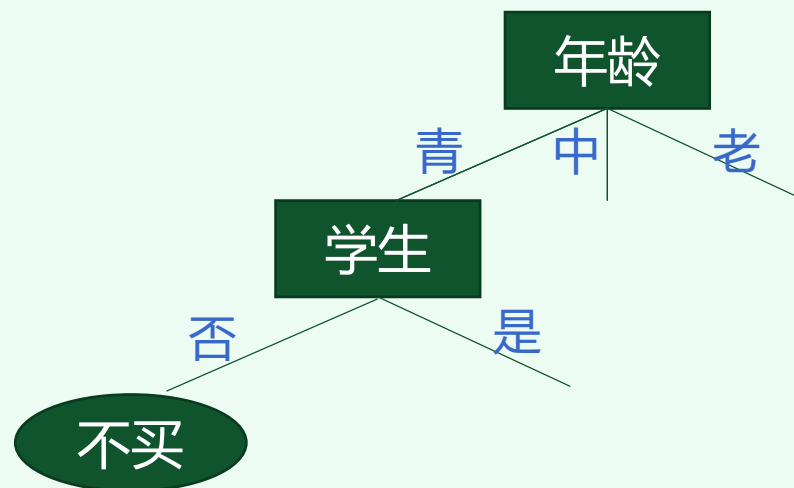
2. 生成 “不买” 类的叶子结点;

3. return; (A出口)

}

$D_{\text{年龄1-学生1}}$

计数	年龄	收入	学生	信誉	归类: 买计算机?
64	青	高	否	良	不买
64	青	高	否	优	不买
128	青	中	否	良	不买



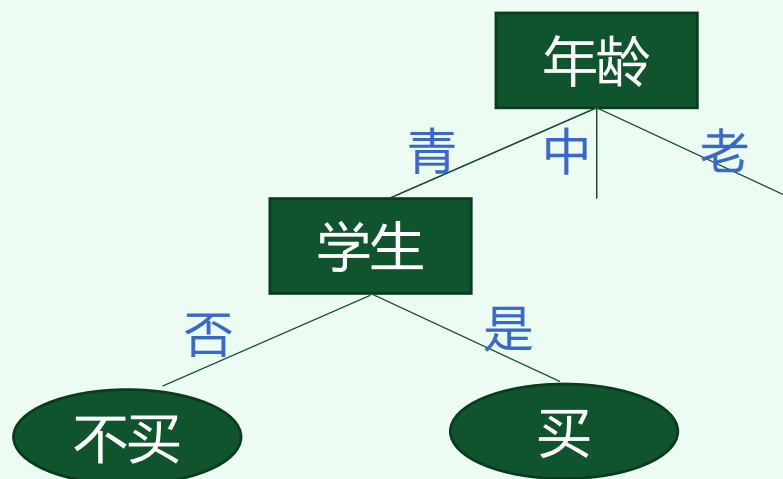
TreeGenerate($D_{\text{年龄1-学生2}}$, $A \setminus \{\text{年龄, 学生}\}$)

1.1.2

- {
- 1. $D_{\text{年龄1-学生2}}$ 中的样本属于同一类 “买”
- 2. 生成 “买” 类的叶子结点;
- 3. return; (A出口)
- }

$D_{\text{年龄1-学生2}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	低	是	良	买
64	青	中	是	优	买





TreeGenerate($D_{\text{年龄2}}$, $A \setminus \{\text{年龄}\}$)

1.2

{

1. $D_{\text{年龄2}}$ 中的样本属于同一类 “买”

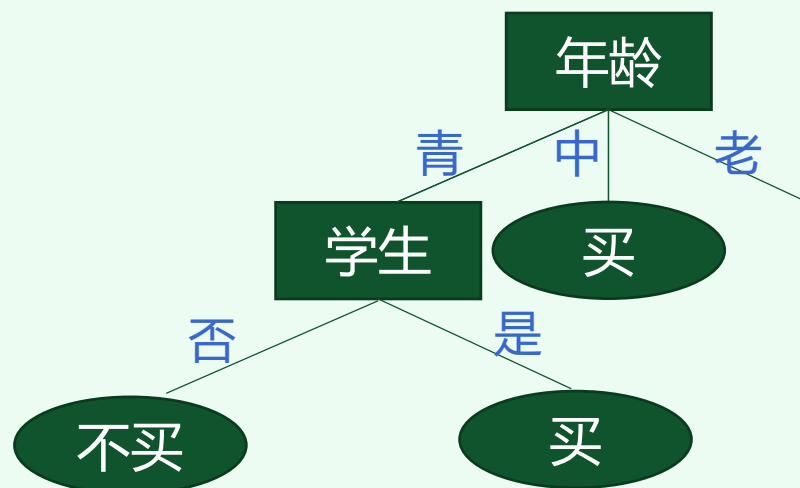
2. 生成 “买” 类的叶子结点;

3. return; (A出口)

}

 $D_{\text{年龄2}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
128	中	高	否	良	买
64	中	低	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买





TreeGenerate($D_{\text{年龄3}}$, $A \setminus \{\text{年龄}\}$)

1.3

{

1. 从 $\{A \setminus \{\text{年龄}\}\}$ 中选出最优划分属性 “信誉”

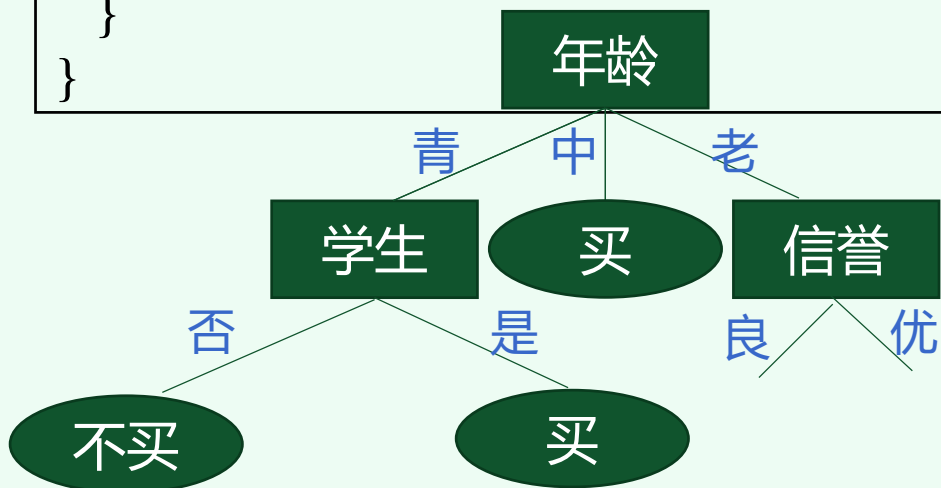
2. 依照 “信誉” 的属性值划分了2个子数据集{

TreeGenerate($D_{\text{年龄3-信誉1}}$, $A \setminus \{\text{年龄、信誉}\}$) ; 1.3.1

TreeGenerate($D_{\text{年龄3-信誉2}}$, $A \setminus \{\text{年龄、信誉}\}$) ; 1.3.2

}

}



$D_{\text{年龄3-信誉1}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
60	老	中	否	良	买
64	老	低	是	良	买
132	老	中	是	良	买

$D_{\text{年龄3-信誉2}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	老	低	是	优	不买
63	老	中	否	优	不买
1	老	中	否	优	买

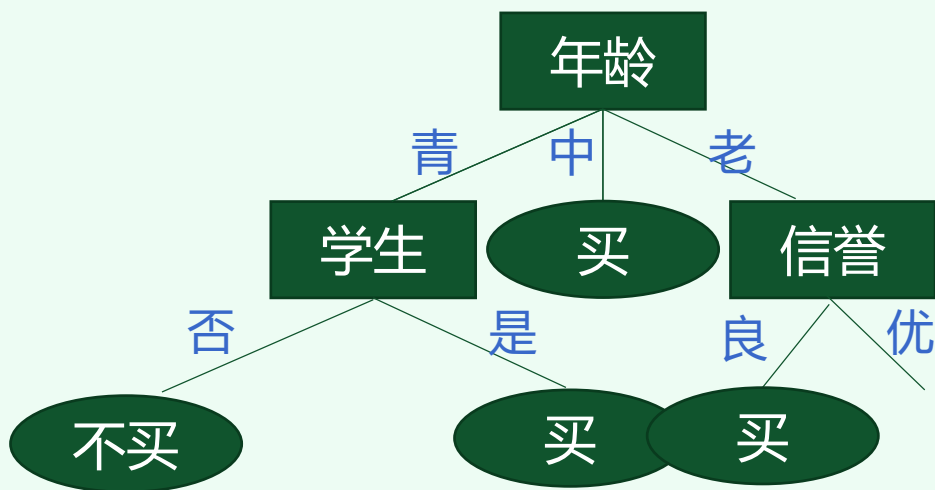


```
TreeGenerate( $D_{\text{年龄3-信誉1}}$ ,  $A \setminus \{\text{年龄, 信誉}\}$ )  
{  
1.  $D_{\text{年龄3-信誉1}}$  中的样本属于同一类 “买”  
2. 生成 “买” 类的叶子结点;  
3. return; (A出口)  
}
```

1.3.1

 $D_{\text{年龄3-信誉1}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
60	老	中	否	良	买
64	老	低	是	良	买
132	老	中	是	良	买





TreeGenerate($D_{\text{年龄3-信誉2}}$, $A \setminus \{\text{年龄, 信誉}\}$)

1.3.2

{
1. 从 $\{A \setminus \{\text{年龄, 信誉}\}\}$ 中选出属性 “学生”

2. 依照 “学生” 的属性值划分了2个子数据集

{
TreeGenerate($D_{\text{年龄3-信誉2-学生1}}$, $A \setminus \{\text{年龄, 信誉, 学生}\}$)

1.3.2.1

TreeGenerate($D_{\text{年龄3-信誉2-学生2}}$, $A \setminus \{\text{年龄, 信誉, 学生}\}$)

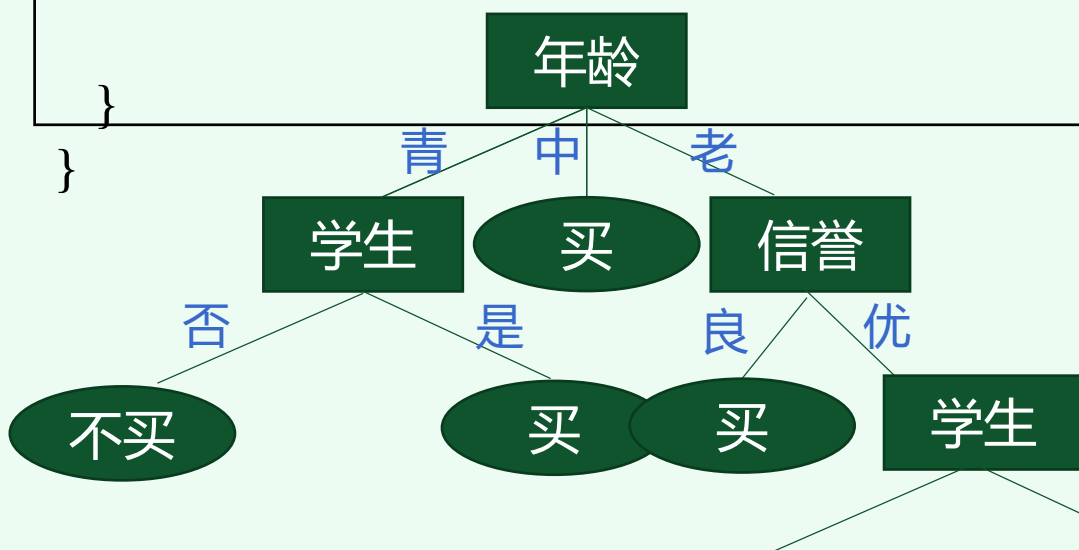
1.3.2.2

$D_{\text{年龄3-信誉2-学生1}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	老	低	是	优	不买

$D_{\text{年龄3-信誉2-学生2}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
63	老	中	否	优	不买
1	老	中	否	优	买



```

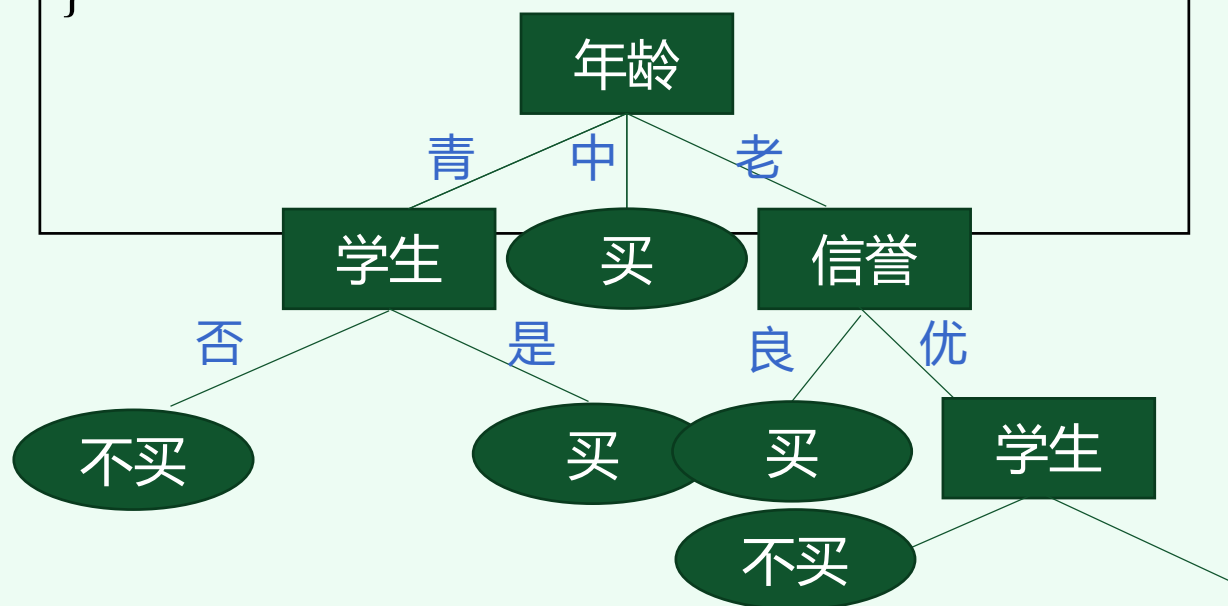
TreeGenerate( $D_{\text{年龄3-信誉2-学生1}}$ ,  $A \setminus \{\text{年龄、信誉、学生}\}$ )
{
  1.  $D_{\text{年龄3-信誉2-学生1}}$  中的样本属于同一类 “不买”
  2. 生成 “不买” 类的叶子结点;
  3. return; (A出口)
}

```

1.3.2.1

 $D_{\text{年龄3-信誉2-学生1}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	老	低	是	优	不买

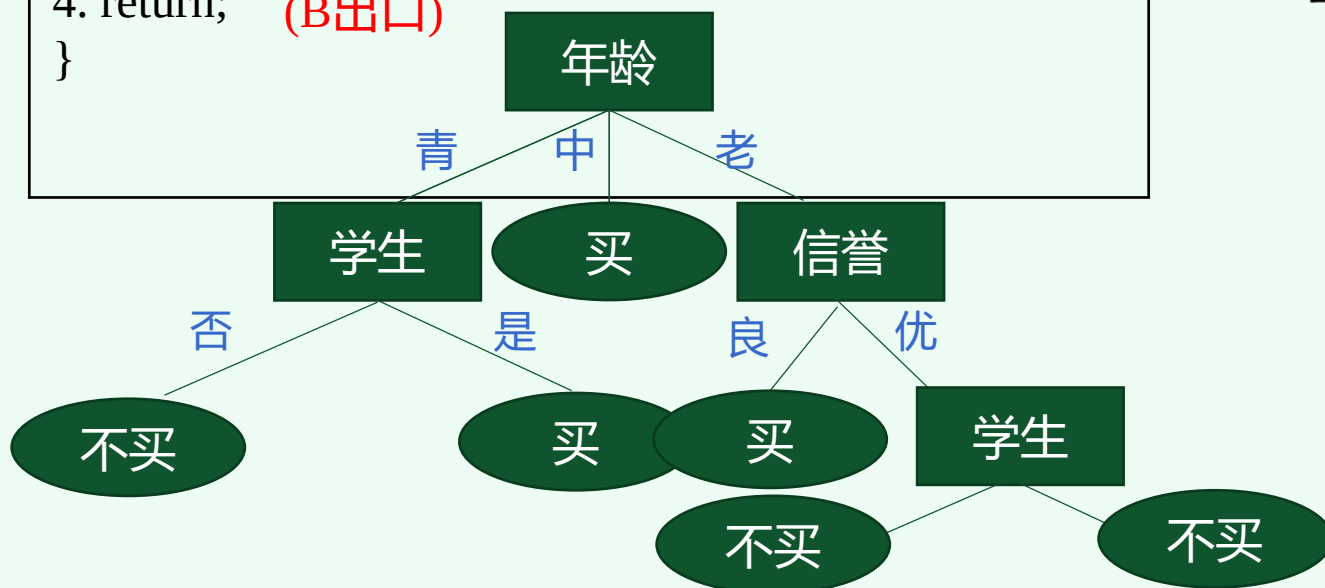


```
TreeGenerate( $D_{\text{年龄3-信誉2-学生2}}$ ,  $A\{\text{年龄、信誉、学生}\}$ )  
{  
1.  $D_{\text{年龄3-信誉2-学生2}}$  在“收入”上的属性相同  
2. 生成叶子结点;  
3. 标记为  $D_{\text{年龄3-信誉2-学生2}}$  中多的类 “不买” ;  
4. return; (B出口)  
}
```

1.3.2.2

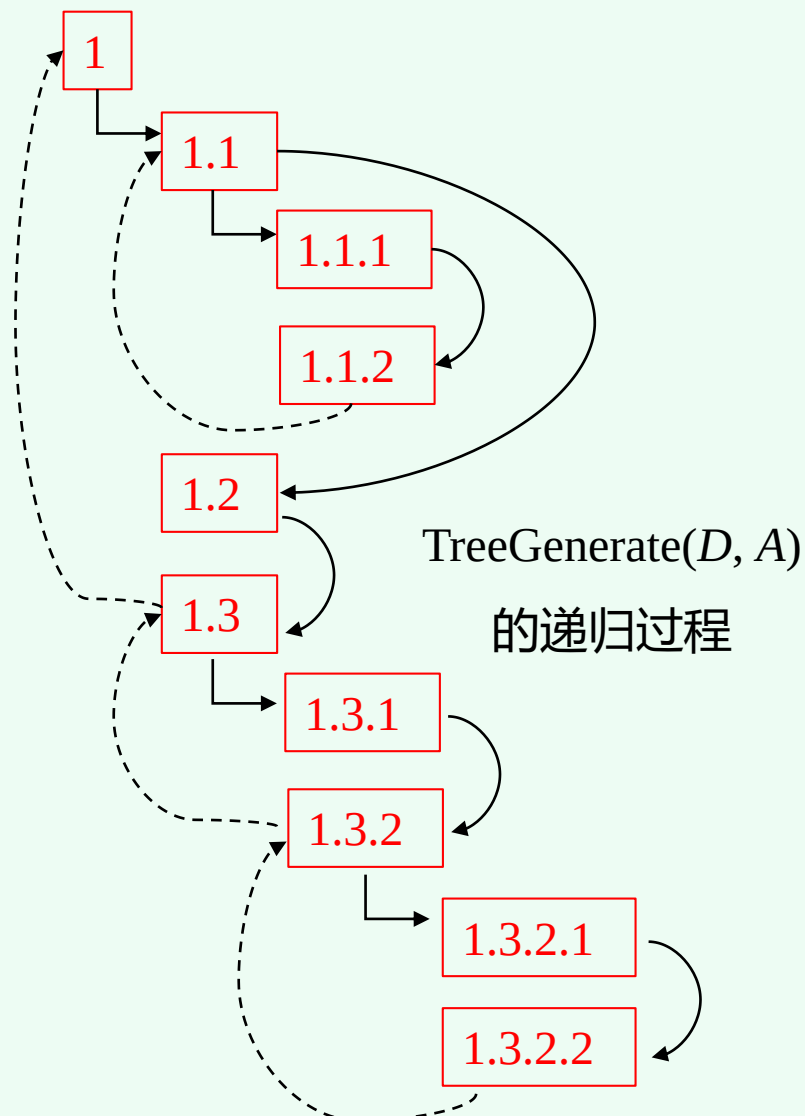
 $D_{\text{年龄3-信誉2-学生2}}$

计数	年龄	收入	学生	信誉	归类：买计算机？
63	老	中	否	优	不买
1	老	中	否	优	买



➤ 三种递归出口分析：

- A：当前结点包含的样本属于同一类，**不需要继续划分**，生成该类叶子结点；
- B：当前属性集为空，或样本在所有属性上取值相同（**无法划分**），按样本最多的类生成该类的叶子结点；（以当前结点的后验概率决定叶子的类）
- C：下一个划分的结点的样本集为空（**不能划分**），按当前结点样本最多的类生成该类的叶子结点；（把父结点的样本分布作为当前结点的先验）

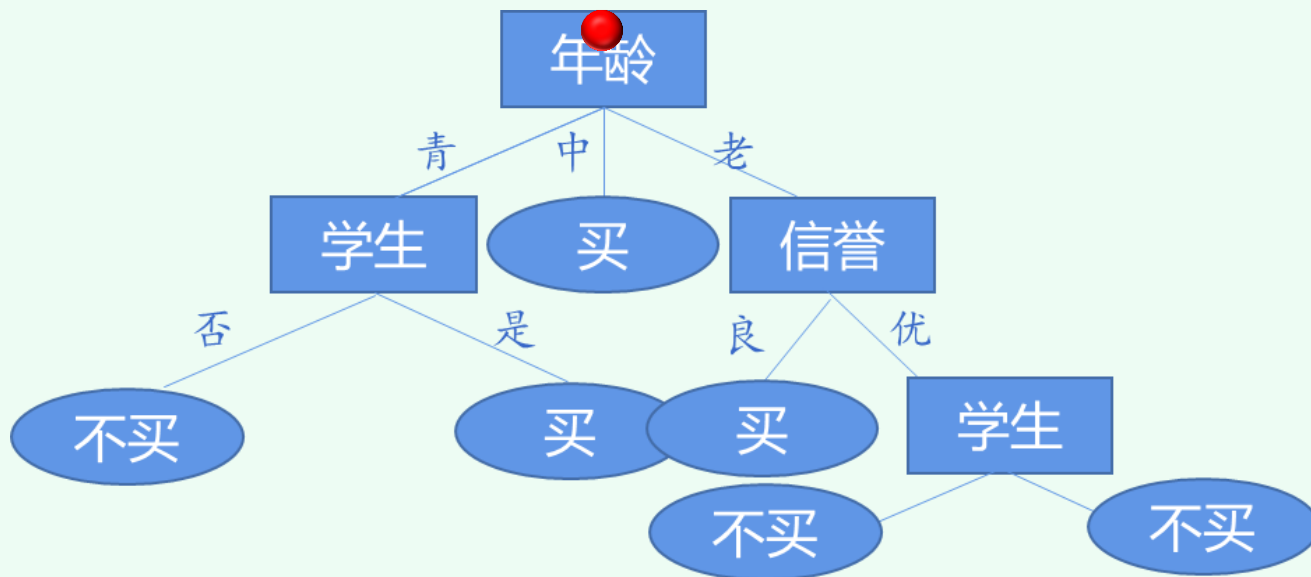


3 依决策树进行决策

- 沿决策树**从根到叶子遍历**，在每个决策结点上对样本属性进行“测试”并转入不同分支，最后到达叶子结点（分类）



测试样本：（年青、收入高、是学生、信誉优）





- 决策树可以是多叉树，也可以是二叉树
- 决策树可以实现多分类
- 决策树是一个非参数、非线性的分类器
- 每次递归的**划分属性**不同，决策树的形态不同



武汉大学
WUHAN UNIVERSITY

02划分选择

partitioning



- 上一节例子中，是在“年龄”、“收入”、“学生”和“信誉”4个属性中每次选择一个进行数据集的划分，从而构建决策树。如果划分的数据集是“最优”的，则被称为**“最佳属性划分”**
- 更多的时候决策树的生成会基于“最佳属性划分”。比如当一个属性的取值不止2个，或者具有连续属性，而需要生成二叉树时，就需要找出最佳的属性进行划分
- “最佳属性划分”和“最佳特征划分”是同一个概念的不同表述。在决策树的构建过程中，关键就是以**结点纯度**作为指标选择最佳的特征（或属性）进行数据划分

1 信息熵纯度

- 在决策树中，一般用**不纯度度量** (impurity measure) 评估分类树划分的优劣。如果样本集 D 的**熵**越小，则可认为其纯度 (purity) 越高。“熵”是度量样本纯度的代表性指标之一
- 样本集 D 中第 i 类样本占比记为 $p_i (i=1,2,\dots,k)$ ，则样本 D 的熵为：

$$\text{Ent}(D) = - \sum_{i=1}^k p_i \log_2 p_i$$

熵越小，则样本集 D 的纯度越高。从样本数据中估计概率（一般都是极大似然估计）而计算出来的熵又称**经验熵**



2 ID3决策树

- ID3 (Iterative Dichotomizer 3, 迭代二分器3) 算法是一种经典的决策树学习算法, 由Quinlan于上世纪70年代提出。ID3 算法的核心是在决策树各个结点上使用**信息增益 (Information Gain)** 进行不纯度度量, 递归地构建决策树

- 假定属性 a 有 V 个可能的取值 $\{a^1, a^2, \dots, a^V\}$, 若使用 a 来对样本集 D 进行划分, 则会产生 V 个分支结点: 其中第 v 个分支结点包含了 D 中所有特征 a 上属性为 a^v 的样本, 记为 D^v
- 属性 a 对样本集 D 的信息增益定义为: 样本集 D 的信息熵与给定属性 a 条件下 D 的条件熵之差, 记为:

$$\mathbf{Gain}(D, a) = \text{Ent}(D) - \text{Ent}(D|a) = \text{Ent}(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} \text{Ent}(D^v)$$

所含样本个数
↙

- ID3模型最优划分属性选择依据:

$$a^* = \underset{a}{\operatorname{argmax}} \mathbf{Gain}(D, a)$$



3. 前面决策树生成中，为什么首先选择“年龄”属性进行划分

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
128	青	中	否	良	不买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买

$$\text{Gain}(D, \text{年龄}) = 0.9537 - \left(\frac{384}{1024} \times 0.9182 + 0 + \frac{384}{1024} \times 0.9156 \right) = 0.2662$$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	青	中	否	良	不买
64	青	低	是	良	买
64	青	中	是	优	买

$$D^v_{\text{年龄}1} = 384 \quad \text{Ent}(D_{\text{年龄}1}) = 0.9182$$

计数	年龄	收入	学生	信誉	归类：买计算机？
128	中	高	否	良	买
64	中	低	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买

$$D^v_{\text{年龄}2} = 256 \quad \text{Ent}(D_{\text{年龄}2}) = 0$$

计数	年龄	收入	学生	信誉	归类：买计算机？
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
132	老	中	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买

$$D^v_{\text{年龄}3} = 384 \quad \text{Ent}(D_{\text{年龄}3}) = 0.9156$$



3. 前面决策树生成中，为什么首先选择“年龄”属性进行划分

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
128	青	中	否	良	不买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买

$$\text{Ent}(D) = -\frac{641}{1024} \log_2 \frac{641}{1024} - \frac{383}{1024} \log_2 \frac{383}{1024} = 0.9537$$

$$\begin{aligned} \text{Gain}(D, \text{学生}) &= 0.9537 - \left(\frac{484}{1024} \times 0.5635 + \frac{540}{1024} \times 0.9761 \right) \\ &= 0.1726 \end{aligned}$$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	高	是	良	买

$$D^{\text{学生}1} = 484 \quad \text{Ent}(D^{\text{学生}1}) = 0.5635$$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
128	青	中	否	良	不买
32	中	中	否	优	买
63	老	中	否	优	不买
1	老	中	否	优	买

$$D^{\text{学生}2} = 540 \quad \text{Ent}(D^{\text{学生}2}) = 0.9761$$

在信息论与概率统计中，熵(entropy)是表示**随机变量不确定性的度量**。设 X 是一个取有限个值的离散随机变量，其概率分布为

$$P(X = x_i) = p_i, \quad i = 1, 2, \dots, n$$

则随机变量 X 的熵定义为

$$H(X) = -\sum_{i=1}^n p_i \log p_i$$

在上式中,若 $p_i = 0$,则定义 $0\log 0=0$ 。通常, 式(5.1)中的对数以2为底或以e为底(自然对数), 这时熵的单位分别称作**比特**(bit)或**纳特**(nat)。由定义可知, 熵只依赖于 X 的分布, 而与 X 的取值无关, 所以也可将 X 的熵记作 $H(p)$ 。

设有随机变量 (X,Y) , 其联合概率分布为

$$P(X = x_i, Y = y_j) = p_{ij}, \quad i = 1, 2, \dots, n; \quad j = 1, 2, \dots, m$$

条件熵 $H(Y|X)$ 表示在已知随机变量 X 的条件下随机变量 Y 的不确定性。随机变量 X 给定的条件下随机变量 Y 的条件熵(conditional entropy) $H(Y|X)$, 定义为 X 给定条件下 Y 的条件概率分布的熵对 X 的数学期望

$$H(Y | X) = \sum_{i=1}^n p_i H(Y | X = x_i) \quad (5.5)$$

这里, $p_i = P(X = x_i)$, $i = 1, 2, \dots, n$

定义 5.2(信息增益) 特征 A 对训练数据集 D 的信息增益 $g(D,A)$, 定义为集合 D 的经验熵 $H(D)$ 与特征 A 给定条件下 D 的经验条件熵 $H(D|A)$ 之差, 即

$$g(D, A) = H(D) - H(D | A) \quad (5.6)$$

一般地, 熵 $H(Y)$ 与条件熵 $H(Y|X)$ 之差称为**互信息**(mutual information)。决策树学习中的信息增益等价于训练数据集中**类与特征的互信息**。

算法 5.1(信息增益的算法)

输入:训练数据集D和特征A;

输出:特征 A对训练数据集 D的信息增益 $g(D,A)$ 。

(1)计算数据集D的经验熵 $H(D)$

$$H(D) = -\sum_{k=1}^K \frac{|C_k|}{|D|} \log_2 \frac{|C_k|}{|D|} \quad (5.7)$$

(2)计算特征A对数据集D的经验条件熵 $H(D|A)$

$$H(D|A) = \sum_{i=1}^n \frac{|D_i|}{|D|} H(D_i) = -\sum_{i=1}^n \frac{|D_i|}{|D|} \sum_{k=1}^K \frac{|D_{ik}|}{|D_i|} \log_2 \frac{|D_{ik}|}{|D_i|} \quad (5.8)$$

(3)计算信息增益

$$g(D,A) = H(D) - H(D|A) \quad (5.9)$$

表 5.1 贷款申请样本数据表

ID	年龄	有工作	有自己的房子	信贷情况	类别
1	青年	否	否	一般	否
2	青年	否	否	好	否
3	青年	是	否	好	是
4	青年	是	是	一般	是
5	青年	否	否	一般	否
6	中年	否	否	一般	否
7	中年	否	否	好	否
8	中年	是	是	好	是
9	中年	否	是	非常好	是
10	中年	否	是	非常好	是
11	老年	否	是	非常好	是
12	老年	否	是	好	是
13	老年	是	否	好	是
14	老年	是	否	非常好	是
15	老年	否	否	一般	否



例5.2 对表5.1所给的训练数据集D，根据信息增益准则选择最优特征。

解 首先计算经验熵 $H(D)$ 。

$$H(D) = -\frac{9}{15} \log_2 \frac{9}{15} - \frac{6}{15} \log_2 \frac{6}{15} = 0.971$$

然后计算各特征对数据集D的信息增益。分别以 A_1, A_2, A_3, A_4 表示年龄、有工作、有自己的房子和信贷情况4个特征，则

$$\begin{aligned} (1) \quad g(D, A_1) &= H(D) - \left[\frac{5}{15} H(D_1) + \frac{5}{15} H(D_2) + \frac{5}{15} H(D_3) \right] \\ &= 0.971 - \left[\frac{5}{15} \left(-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \right) + \frac{5}{15} \left(-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \right) + \frac{5}{15} \left(-\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \right) \right] \\ &= 0.971 - 0.888 = 0.083 \end{aligned}$$

这里 D_1, D_2, D_3 分别是D中 A_1 取值为青年、中年和老年的样本子集。类似地，



$$\begin{aligned}(2) \quad g(D, A_2) &= H(D) - \left[\frac{5}{15} H(D_1) + \frac{10}{15} H(D_2) \right] \\ &= 0.971 - \left[\frac{5}{15} \times 0 + \frac{10}{15} \left(-\frac{4}{10} \log_2 \frac{4}{10} - \frac{6}{10} \log_2 \frac{6}{10} \right) \right] = 0.324\end{aligned}$$

$$\begin{aligned}(3) \quad g(D, A_3) &= 0.971 - \left[\frac{6}{15} \times 0 + \frac{9}{15} \left(-\frac{3}{9} \log_2 \frac{3}{9} - \frac{6}{9} \log_2 \frac{6}{9} \right) \right] \\ &= 0.971 - 0.551 = 0.420\end{aligned}$$

$$(4) \quad g(D, A_4) = 0.971 - 0.608 = 0.363$$

最后，比较各特征的信息增益值。由于特征 A_3 (有自己的房子) 的信息增益值最大，所以选择特征 A_3 作为最优特征。



利用上述计算结果，由于特征 A_3 (有自己的房子) 的信息增益值最大，所以选择特征 A_3 作为 **根结点** 的特征。它将训练数据集 D 划分为两个子集 $D(A_3 \text{ 取值为“是”})$ 和 $D(A_3 \text{ 取值为“否”})$ 。由于 D_1 只有同一类的样本点，所以它成为一个叶结点，结点的类标记为“是”。

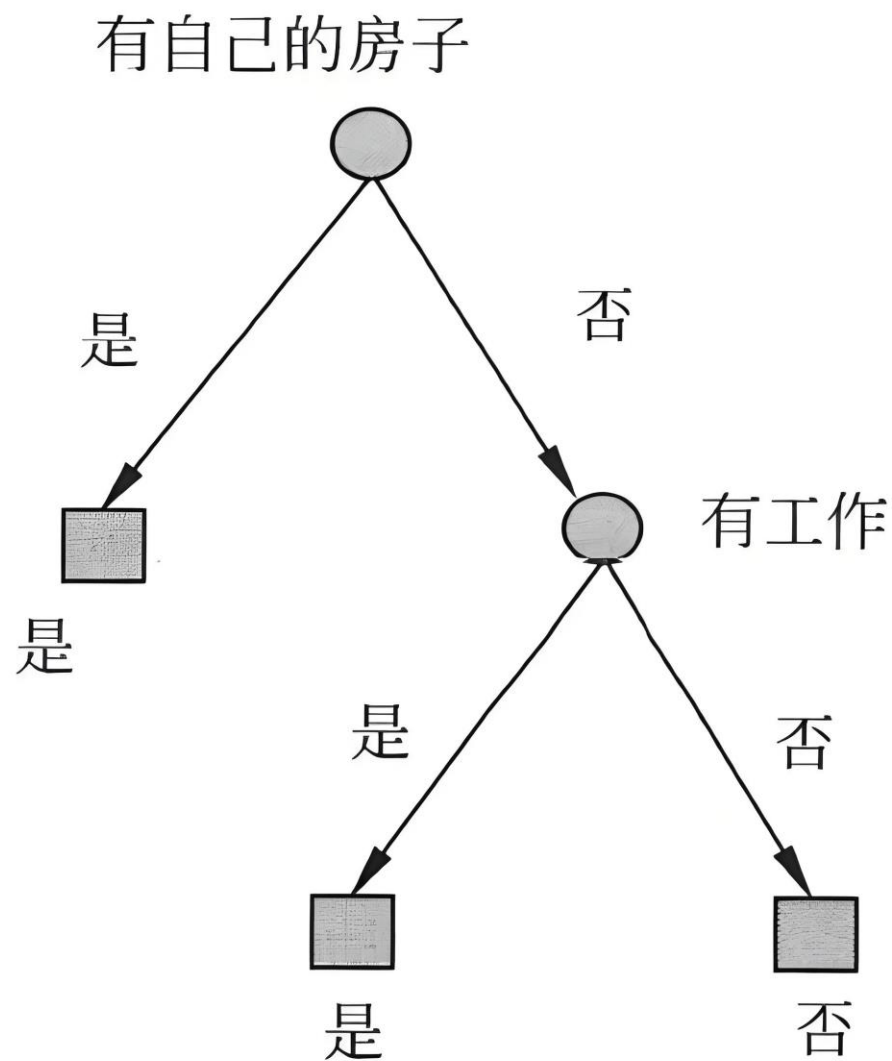
对 D_2 则需从特征 A_1 (年龄)， A_2 (有工作) 和 A_4 (信贷情况) 中选择新的特征计算各个特征的信息增益：

$$g(D_2, A_1) = H(D_2) - H(D_2 | A_1) = 0.918 - 0.667 = 0.251$$

$$g(D_2, A_2) = H(D_2) - H(D_2 | A_2) = 0.918$$

$$g(D_2, A_4) = H(D_2) - H(D_2 | A_4) = 0.474$$

选择信息增益最大的特征 A_2 (有工作) 作为结点的特征。由于有两个可能取值，从这一结点引出两个子结点：一个对应“是”(有工作)的子结点，包含3个样本，它们属于同一类，所以这是一个叶结点，类标记为“是”；另一个是对应“否”(无工作)的子结点，包含6个样本，它们也属于同一类，所以这这也是一个叶结点，类标记为“否”这样生成一棵如图5.5所示的决策树。该决策树**只用了两个特征**(有两个内部结点)。



例5.3中构建的决策树



ID3决策树的实现

```
import pandas as pd
import numpy as np
from math import log2

# 计算信息熵
def entropy(target_col):
    elements, counts = np.unique(target_col,
    return_counts=True)
    entropy = sum([- (count / len(target_col)) *
log2(count / len(target_col)) for count in counts])
    return entropy
```




```
# 计算信息增益
def info_gain(data, split_attribute_name, target_name="Buy
Computer?"):
    # 计算数据集的信息熵
    total_entropy = entropy(data[target_name])

    # 获取属性值的唯一值
    values = data[split_attribute_name].unique()

    # 计算每个属性值的信息熵
    weighted_entropy = 0
    for value in values:
        subset = data[data[split_attribute_name] == value]
        weight = len(subset) / len(data)
        weighted_entropy += weight *
entropy(subset[target_name])

    # 计算信息增益
    gain = total_entropy - weighted_entropy
    return gain
```



```
# 选择最优的属性
def best_attribute(data, attributes, target_name="Buy
Computer?"):
    gains = [info_gain(data, attr, target_name) for attr in
attributes]
    best_attr = attributes[np.argmax(gains)]
    return best_attr

# 构建决策树
def id3(data, attributes, target_name="Buy Computer?"):
    # 如果所有样本的目标值相同, 则返回该值
    if len(np.unique(data[target_name])) == 1:
        return np.unique(data[target_name])[0]

    # 如果没有属性可以选择, 则返回最多的目标值
    if not attributes:
        return data[target_name].mode()[0]
```



```
# 选择最佳属性
best_attr = best_attribute(data, attributes, target_name)
tree = {best_attr: {}}

# 创建子树
for value in np.unique(data[best_attr]):
    subset = data[data[best_attr] == value]
    subtree = id3(subset, [attr for attr in attributes
if attr != best_attr], target_name)
    tree[best_attr][value] = subtree

return tree
```



创建数据集并包含样本数量

```
data = pd.DataFrame({
    'Age': ['青'] * 64 + ['青'] * 64 + ['中'] * 128 + ['老'] * 60 + ['老'] * 64 + ['老'] *
    64 + ['中'] * 64 + [
        '青'] * 128 + ['青'] * 64 + ['老'] * 132 + ['青'] * 64 + ['中'] * 32 + ['中'] * 32 +
        ['老'] * 63 + ['老'] * 1,
    'Income': ['高'] * 64 + ['高'] * 64 + ['高'] * 128 + ['中'] * 60 + ['低'] * 64 + ['低']
    * 64 + ['低'] * 64 + [
        '中'] * 128 + ['低'] * 64 + ['中'] * 132 + ['中'] * 64 + ['中'] * 32 + ['高'] * 32 +
        ['中'] * 63 + ['中'] * 1,
    'Student': ['否'] * 64 + ['否'] * 64 + ['否'] * 128 + ['否'] * 60 + ['是'] * 64 + ['是']
    * 64 + ['是'] * 64 + [
        '否'] * 128 + ['是'] * 64 + ['是'] * 132 + ['是'] * 64 + ['否'] * 32 + ['是'] * 32 +
        ['否'] * 63 + ['否'] * 1,
    'Credit': ['良'] * 64 + ['尤'] * 64 + ['良'] * 128 + ['良'] * 60 + ['良'] * 64 + ['优']
    * 64 + ['优'] * 64 + [
        '良'] * 128 + ['良'] * 64 + ['良'] * 132 + ['优'] * 64 + ['优'] * 32 + ['良'] * 32 +
        ['优'] * 63 + ['优'] * 1,
    'Buy Computer?': ['不买'] * 64 + ['不买'] * 64 + ['买'] * 128 + ['买'] * 60 + ['买'] *
    64 + ['不买'] * 64 + [
        '买'] * 64 + ['不买'] * 128 + ['买'] * 64 + ['买'] * 132 + ['买'] * 64 + ['买'] * 32
    + ['买'] * 32 + [
        '不买'] * 63 + ['买'] * 1
})
```

获取属性列表

```
attributes = ['Age', 'Income', 'Student', 'Credit']
```

构建决策树

```
decision_tree = id3(data, attributes)
```

```
print(decision_tree)
```

3 C4.5决策树

- **信息增益准则对可取值数目较多（分支较多）的属性有所偏好**，为减少这种偏好可能带来的不利影响，Quinlan在1993年提出了著名的**C4.5决策树**，使用**增益率（Gain ratio）**来选择最优划分属性：

$$\text{Gain_ratio}(D,a) = \frac{\text{Gain}(D,a)}{\text{IV}(a)}, \quad \text{IV}(a) = - \sum_{v=1}^V \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|}$$

- 其中 $\text{IV}(a)$ 称为属性 a 的**固有价值**（intrinsic value）。属性 a 可能取值数目越大，一般 $\text{IV}(a)$ 也会越大，作为分母平衡一下信息增益



➤ C4.5算法最优划分属性选择依据：

$$a^* = \underset{a}{\operatorname{argmax}} \text{Gain_ratio}(D, a)$$

注：C4.5并不一定直接选择信息增益率最大的属性，而是在候选特征中找出信息增益高于平均水平的属性，然后在这些属性中选择信息增益率最高的那个。**因为该准则对取值数目较少的属性有偏好。**



4 CART决策树

- **分类与回归树** (Classification And Regression Tree, CART) 模型由Breiman等人于1984年提出，是一种应用广泛的决策树模型。CART既可以用于处理**分类**问题，也可以用于处理**回归**问题
- CART 决策树是**二叉树**。内部结点特征的取值为“是”和“否”，左分支是取值为“是”的分支，右分支是“否”分支。这样的决策树等价于递归地二分每个特征，将样本空间划分为有限个单元，并在这些单元上确定预测的概率分布

4 CART决策树

- 分类与回归树 (Classification And Regression Tree, CART) 模型由Breiman等人于1984年提出, 是一种应用广泛的决策树模型。CART既可以用于处理分类问题, 也可以用于处理回归问题



- 里奥·布莱曼 (LeoBreiman), 世界著名应用统计学家, 先后在UCLA和UCBerkeley任教, 是机器学习领域的主要领导者之一。布莱曼系统提出了分类与回归树 (**CART**) 方法, 并对集成学习做出重大贡献, 在决策树基础上提出了**随机森林** (Random Forests) 方法。



Leo Breiman
(1928-2005)

- CART决策树使用**基尼指数** (Gini index) 度量数据纯度
- 样本集 D 中第 i 类样本占比记为 $p_i (i=1,2,\dots,k)$ 。样本集的纯度用**基尼值度量**，该值越小，样本集 D 的纯度越高，表示为：

$$\text{Gini}(D) = \sum_{i=1}^k \sum_{i' \neq i} p_i p_{i'} = 1 - \sum_{i=1}^k p_i^2$$

- 采用属性 a 划分的**基尼指数**表示为：

$$\text{Gini_index}(D, a) = \sum_{v=1}^V \frac{|D^v|}{|D|} \text{Gini}(D^v)$$

➤ CART算法最优划分属性选择的依据是最小化基尼指数:

$$a^* = \underset{a}{\operatorname{argmin}} \text{Gini_index}(D, a)$$



g. 试计算这两种划分的基尼指数，看哪一种划分更优

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	青	中	否	良	不买
64	青	低	是	良	买
64	青	中	是	优	买

计数	年龄	收入	学生	信誉	归类：买计算机？
128	中	高	否	良	买
64	中	低	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买

计数	年龄	收入	学生	信誉	归类：买计算机？
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
132	老	中	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买

计数	年龄	收入	学生	信誉	归类：买计算机？
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	高	是	良	买

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
128	青	中	否	良	不买
32	中	中	否	优	买
63	老	中	否	优	不买
1	老	中	否	优	买



算法5.6(CART 生成算法)

输入:训练数据集 D , 停止计算的条件;

输出:CART 决策树,

根据训练数据集, 从根结点开始, 递归地对每个结点进行以下操作, 构建二叉决策树

(1)设结点的训练数据集为 D , 计算现有特征对该数据集的基尼指数。此时, 对每一个特征 A , 对其可能取的每个值 a , 根据样本点对 $A = a$ 的测试为“是”或“否”将 D 分割成 D_1 和 D_2 两部分,) 计算 $A = a$ 时的基尼指数。

(2)在所有可能的特征 A 以及它们所有可能的切分点 a 中, 选择基尼指数最小的特征及其对应的切分点作为最优特征与最优切分点。依最优特征与最优切分点, 从现结点生成两个子结点, 将训练数据集依特征分配到两个子结点中去。

(3)对两个子结点递归地调用(1),(2), 直至满足停止条件。

(4)生成CART决策树。

算法**停止计算的条件**是结点中的样本个数小于预定阈值, 或样本集的基尼指数小于预定阈值(样本基本属于同一类), 或者没有更多特征。



例5.4 根据表5.1 所给训练数据集，应用CART 算法生成决策树。

解 首先计算各特征的基尼指数,选择最优特征以及其最优切分点。仍采用例5.2的记号，分别以 A_1, A_2, A_3, A_4 表示年龄、有工作、有自己的房子和信贷情况4个特征，并以 1,2,3表示年龄的值为青年、中年和老年，以1,2表示有工作和有自己的房子的值为是和否，以1, 2, 3表示信贷情况的值为非常好、好和一般。

求特征 A_1 的基尼指数:

$$\text{Gini}(D, A_1 = 1) = \frac{5}{15} \left(2 \times \frac{2}{5} \times \left(1 - \frac{2}{5} \right) \right) + \frac{10}{15} \left(2 \times \frac{7}{10} \times \left(1 - \frac{7}{10} \right) \right) = 0.44$$

$$\text{Gini}(D, A_1 = 2) = 0.48$$

$$\text{Gini}(D, A_1 = 3) = 0.44$$



由于 $\text{Gini}(D, A_1 = 1)$ 和 $\text{Gini}(D, A_1 = 3)$ 相等，且最小，所以 $A_1 = 1$ 和 $A_1 = 3$ 都可以选作 A_1 的最优切分点。

求特征 A_2 和 A_3 的基尼指数：

$$\text{Gini}(D, A_2 = 1) = 0.32$$

$$\text{Gini}(D, A_3 = 1) = 0.27$$

由于 A_2 和 A_3 只有一个切分点，所以它们就是最优切分点。



求特征 A_4 的基尼指数：

$$\text{Gini}(D, A_4 = 1) = 0.36$$

$$\text{Gini}(D, A_4 = 2) = 0.47$$

$$\text{Gini}(D, A_4 = 3) = 0.32$$

$\text{Gini}(D, A_4 = 3)$ 最小，所以 $A_4 = 3$ 就是 A_4 的最优切分点。

在 A_1, A_2, A_3, A_4 几个特征中， $\text{Gini}(D, A_3 = 1) = 0.27$ 最小，所以选择特征 A_3 为最优特征， $A_3 = 1$ 为其最优切分点。于是根结点生成两个子结点，一个是叶结点。对另一个结点继续使用以上方法在 A_1, A_2, A_4 中选择最优特征及其最优切分点，结果是 $A_2 = 1$ 。依此计算得知，所得结点都是叶结点。

对于本问题，按照 CART 算法所生成的决策树与按照ID3算法所生成的决策树**完全一致**。

5 连续属性与缺失属性的处理 (西瓜书p83-88)

- 若属性取值是连续值，不能直接根据数值进行结点划分。需要使用**连续属性离散化技术**。比如C4.5决策树采用**二分法**(bi-partition)对连续属性进行处理

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
128	青	中	否	良	不买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买



计数	年龄	收入	学生	信誉	归类：买计算机？
64	18	高	否	良	不买
64	19	高	否	优	不买
128	25	高	否	良	买
60	45	中	否	良	买
64	50	低	是	良	买
64	55	低	是	优	不买
64	28	低	是	优	买
128	20	中	否	良	不买
64	21	低	是	良	买
132	55	中	是	良	买
64	22	中	是	优	买
32	32	中	否	优	买
32	35	高	是	良	买
63	60	中	否	优	不买
1	65	中	否	优	买



- 可以根据样本中属性 a 的连续取值从小到大排序 $\{a^1, a^2, a^3, \dots, a^n\}$ ，基于划分点 t 将样本集分为两个子集（属性 a 的取值不大于 t 和大于 t ）。可以按照**中位点**的方式设置 $n-1$ 个候选划分点 $T_a = \{t_1, t_2, \dots\}$ ，再通过计算每一个划分点二分后的**信息增益**，找到使信息增益最大的候选点作为划分点，将属性 a 以划分点为界离散化
- 若当前结点划分属性为连续属性，该属性还可以作为其后代结点的划分属性**继续使用**



“年龄”属性的候选划分点集合包含14个候选值： $T_{\text{年龄}} = \{18.5, 19.5, 20.5, 21.5, 23.5, 26.5, 30.0, 33.5, 40.0, 47.5, 52.5, 55.0, 57.5, 62.5\}$ ，分别对应的信息增益为 $\{0.0937, 0.1997, \mathbf{0.4684}, 0.2599, 0.1601, 0.0496, 0.0199, 0.0101, 0.0035, 0.0004, 0.0134, 0.0855, 0.0855, 0.0006\}$ ，由此可以计算出属性“年龄”的信息增益为0.4684，对应于划分点20.5

计数	年龄	收入	学生	信誉	归类：买计算机？
64	18	高	否	良	不买
64	19	高	否	优	不买
128	25	高	否	良	买
60	45	中	否	良	买
64	50	低	是	良	买
64	55	低	是	优	不买
64	28	低	是	优	买
128	20	中	否	良	不买
64	21	低	是	良	买
132	55	中	是	良	买
64	22	中	是	优	买
32	32	中	否	优	买
32	35	高	是	良	买
63	60	中	否	优	不买
1	65	中	否	优	买



计数	年龄	收入	学生	信誉	归类：买计算机？
64		高	否	良	不买
64		高	否	优	不买
128		高	否	良	买
60		中	否	良	买
64		低	是	良	买
64		低	是	优	不买
64		低	是	优	买
128		中	否	良	不买
64		低	是	良	买
132		中	是	良	买
64		中	是	优	买
32		中	否	优	买
32		高	是	良	买
63		中	否	优	不买
1		中	否	优	买

以划分点为界离散化

➤ 如何在属性值缺失的情况下计算样本纯度?

• 以属性不缺失的样本计算信息增益，乘以无缺失样本占该属性所有样本的比例作为权重系数

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
128	青	中	否	良	不买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买



计数	年龄	收入	学生	信誉	归类：买计算机？
64	■	高	否	良	不买
64	青	高	否	■	不买
128	中	高	否	良	买
60	■	中	否	良	买
64	老	低	■	良	买
64	老	低	是	优	不买
64	■	低	是	优	买
128	青	■	否	良	不买
64	青	低	是	■	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	■	否	优	买
32	中	高	是	良	买
63	老	中	■	优	不买
1	老	中	否	优	买



计数	年龄	收入	学生	信誉	归类：买计算机？
64	■	高	否	良	不买
64	青	高	否	■	不买
128	中	高	否	良	买
60	■	中	否	良	买
64	老	低	■	良	买
64	老	低	是	优	不买
64	■	低	是	优	买
128	青	■	否	良	不买
64	青	低	是	■	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	■	否	优	买
32	中	高	是	良	买
63	老	中	■	优	不买
1	老	中	否	优	买

$$\tilde{D}^v_{\text{年龄}_1} = 320$$

$$\text{Ent}(\tilde{D}_{\text{年龄}_1}) = - \frac{128}{320} \log_2 \frac{128}{320} - \frac{192}{320} \log_2 \frac{192}{320} = 0.9706$$

$$\tilde{D}^v_{\text{年龄}_2} = 192$$

$$\text{Ent}(\tilde{D}_{\text{年龄}_2}) = 0$$

注意这些系数和前面例子的差别

$$\tilde{D}^v_{\text{年龄}_3} = 324$$

$$\text{Ent}(\tilde{D}_{\text{年龄}_3}) = - \frac{197}{324} \log_2 \frac{197}{324} - \frac{127}{324} \log_2 \frac{127}{324} = 0.9661$$

$$\text{Ent}(\tilde{D}) = - \frac{517}{836} \log_2 \frac{517}{836} - \frac{319}{836} \log_2 \frac{319}{836} = 0.9591$$

$$\text{Gain}(\tilde{D}, \text{年龄}) = 0.9591 - \left(\frac{320}{836} \times 0.9706 + 0 + \frac{324}{836} \times 0.9661 \right) = 0.2248$$

$$\text{Gain}(\mathbf{D}, \text{年龄}) = \rho * \text{Gain}(\tilde{D}, \text{年龄}) = \frac{836}{1024} \times 0.2248 = 0.1835$$



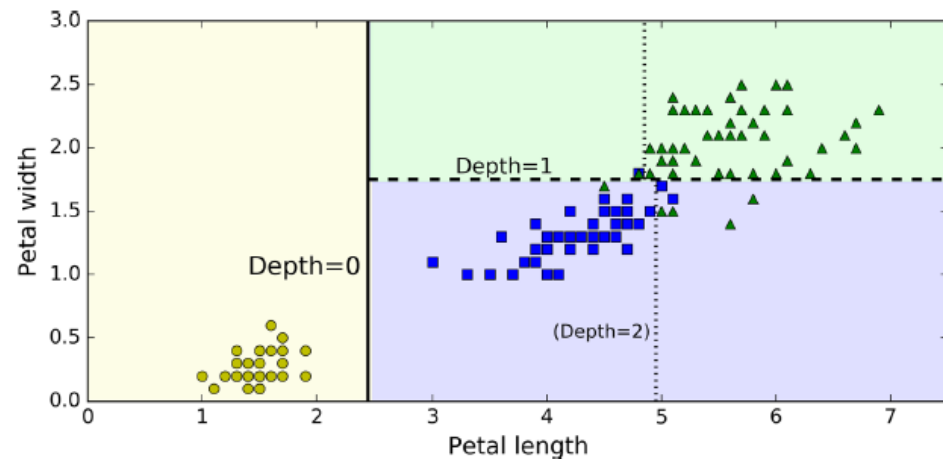
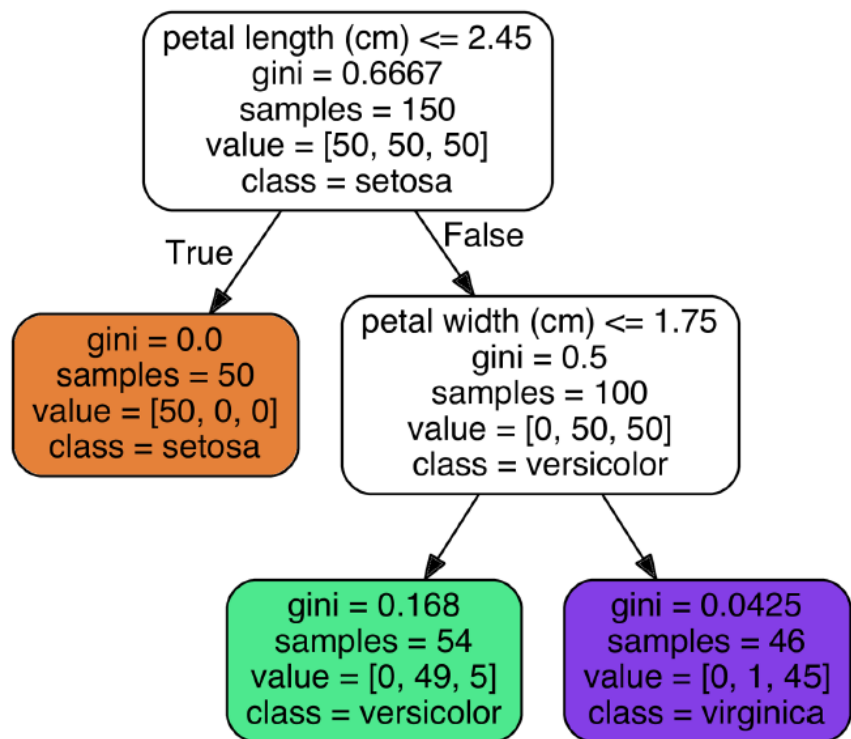
```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
```

```
iris = load_iris()
X = iris.data[:, 2:] # petal length and width
y = iris.target
```

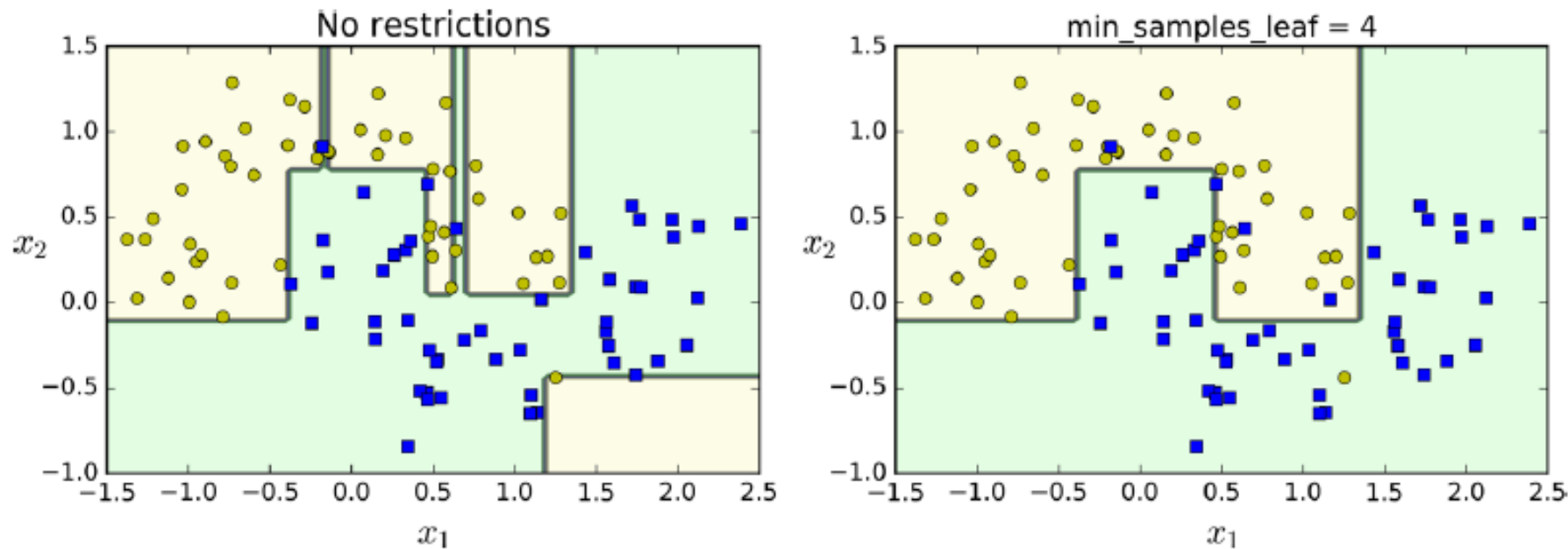
```
tree_clf = DecisionTreeClassifier(max_depth=2)
tree_clf.fit(X, y)
```

```
from sklearn.tree import export_graphviz
```

```
export_graphviz(
    tree_clf,
    out_file=image_path("iris_tree.dot"),
    feature_names=iris.feature_names[2:],
    class_names=iris.target_names,
    rounded=True,
    filled=True
)
```



决策边界



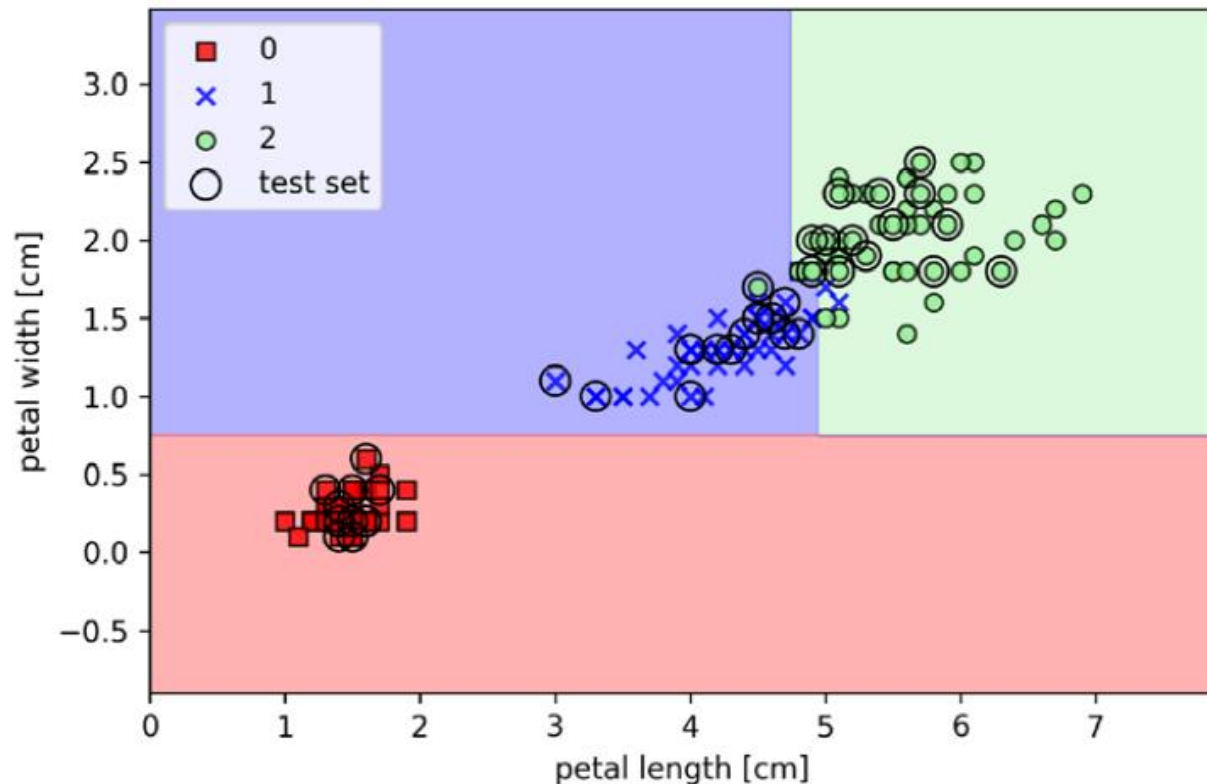
左图：过拟合。右图：使用超参`min_samples_leaf`进行正则化

其他模型正则化参数：

`min_samples_split` （分裂前节点必须有的最小样本数） ，
`min_samples_leaf` （叶节点必须有的最小样本数量） ，
`min_weight_fraction_leaf` （叶节点必须有的最小样本的加权占比） ，
`max_leaf_nodes` （最大叶节点数量） ，
`Max_features` （分裂每个节点评估的最大特征数量）

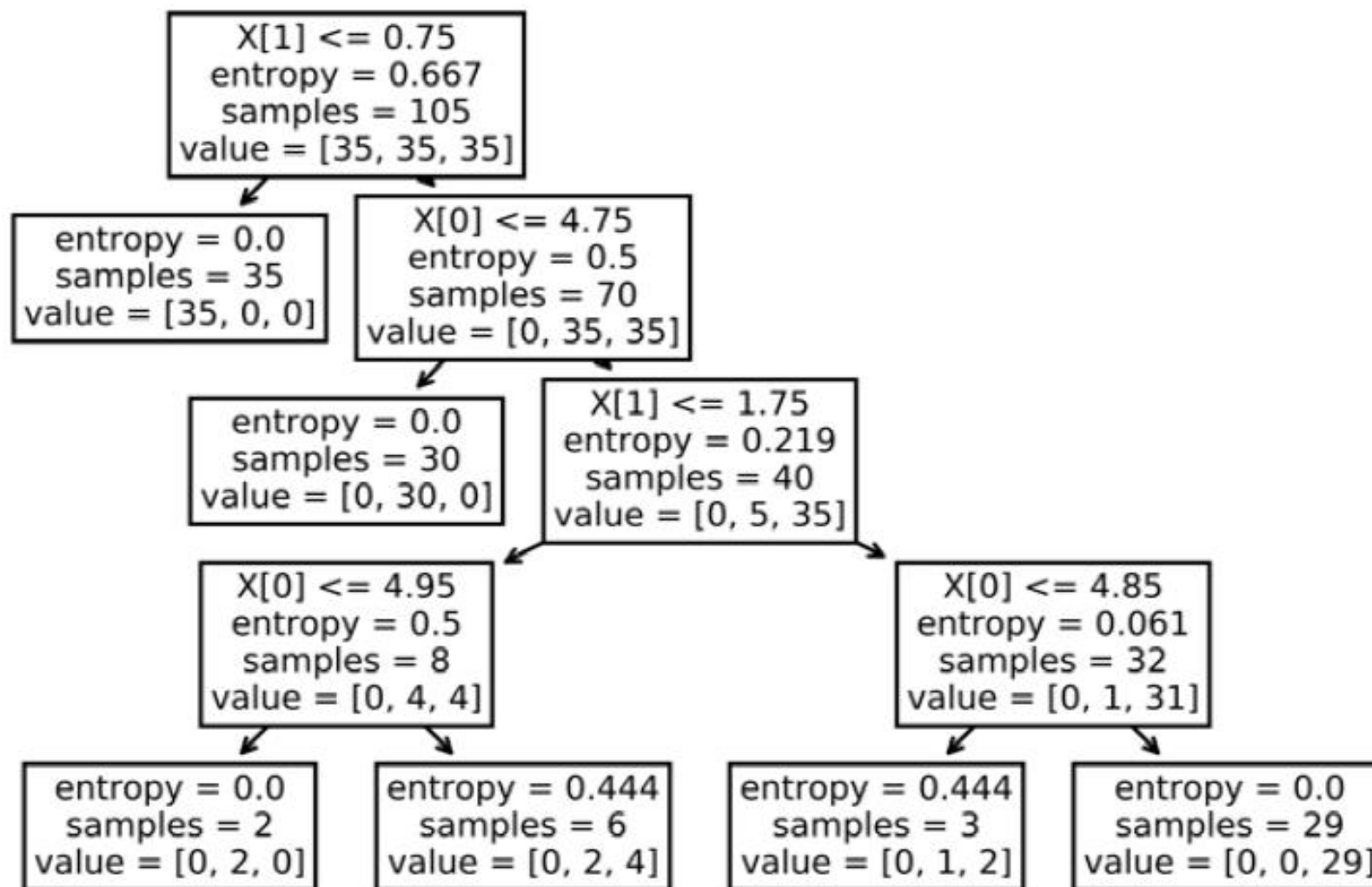


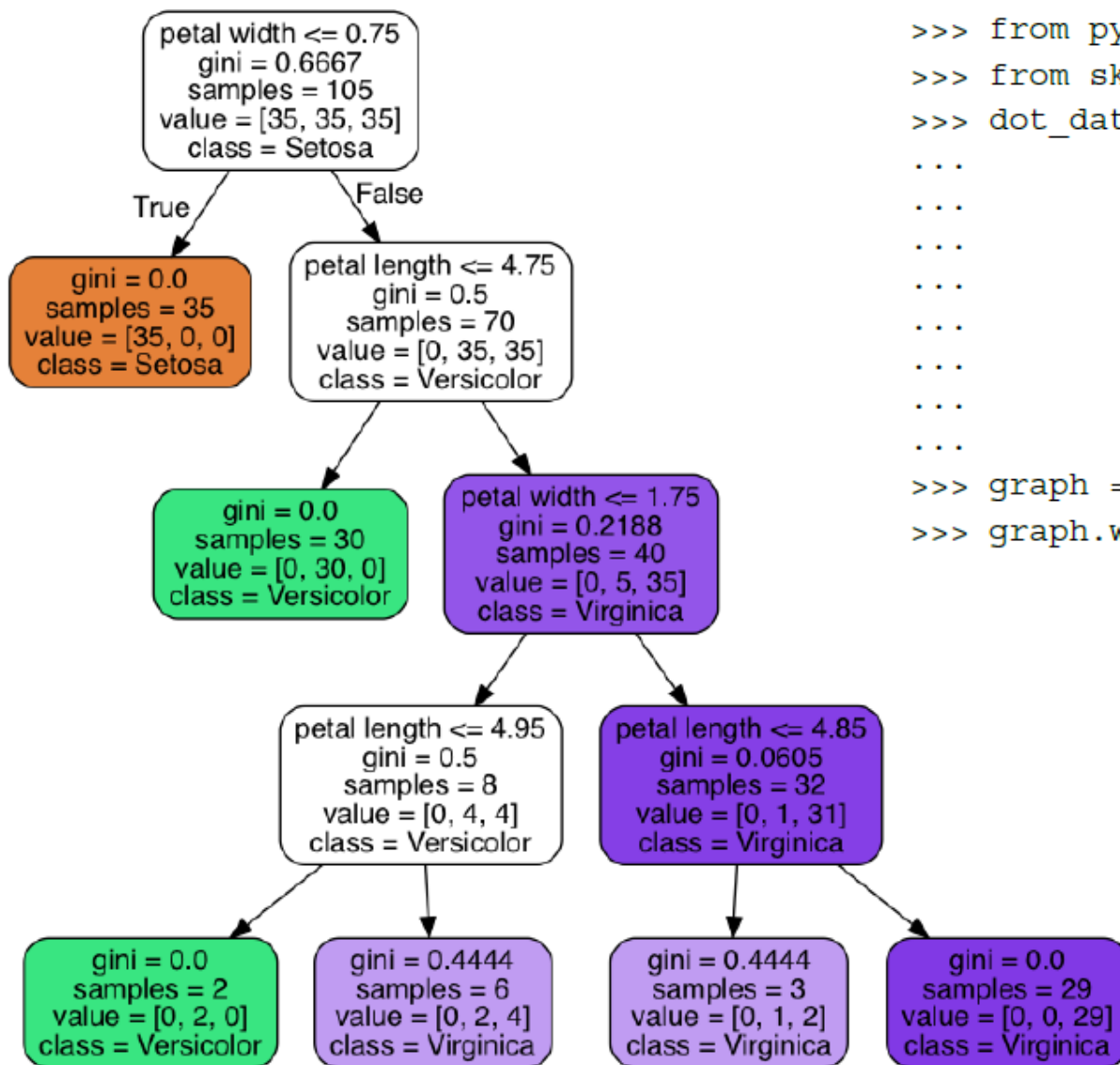
```
>>> from sklearn.tree import DecisionTreeClassifier
>>> tree = DecisionTreeClassifier(criterion='gini',
...                               max_depth=4,
...                               random_state=1)
>>> tree.fit(X_train, y_train)
>>> X_combined = np.vstack((X_train, X_test))
>>> y_combined = np.hstack((y_train, y_test))
>>> plot_decision_regions(X_combined,
...                       y_combined,
...                       classifier=tree,
...                       test_idx=range(105, 150))
>>> plt.xlabel('petal length [cm]')
>>> plt.ylabel('petal width [cm]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```





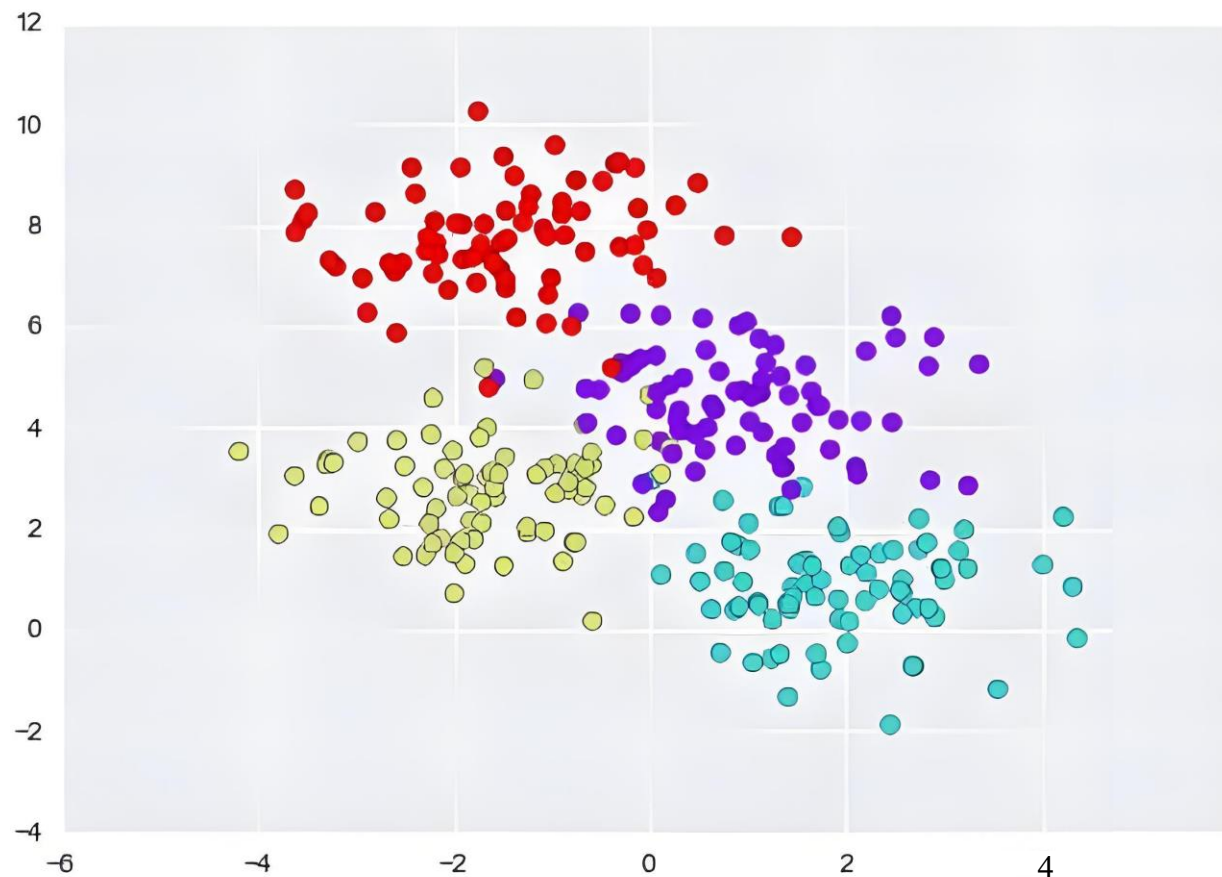
```
>>> from sklearn import tree  
>>> tree.plot_tree(tree_model)  
>>> plt.show()
```





```

>>> from pydotplus import graph_from_dot_data
>>> from sklearn.tree import export_graphviz
>>> dot_data = export_graphviz(tree_model,
...                             filled=True,
...                             rounded=True,
...                             class_names=['Setosa',
...                                           'Versicolor',
...                                           'Virginica'],
...                             feature_names=['petal length',
...                                           'petal width'],
...                             out_file=None)
>>> graph = graph_from_dot_data(dot_data)
>>> graph.write_png('tree.png')
  
```



生成用于实验的散点数据

```
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=300, centers=4,
                  random_state=0, cluster_std=1.0)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='rainbow');
```



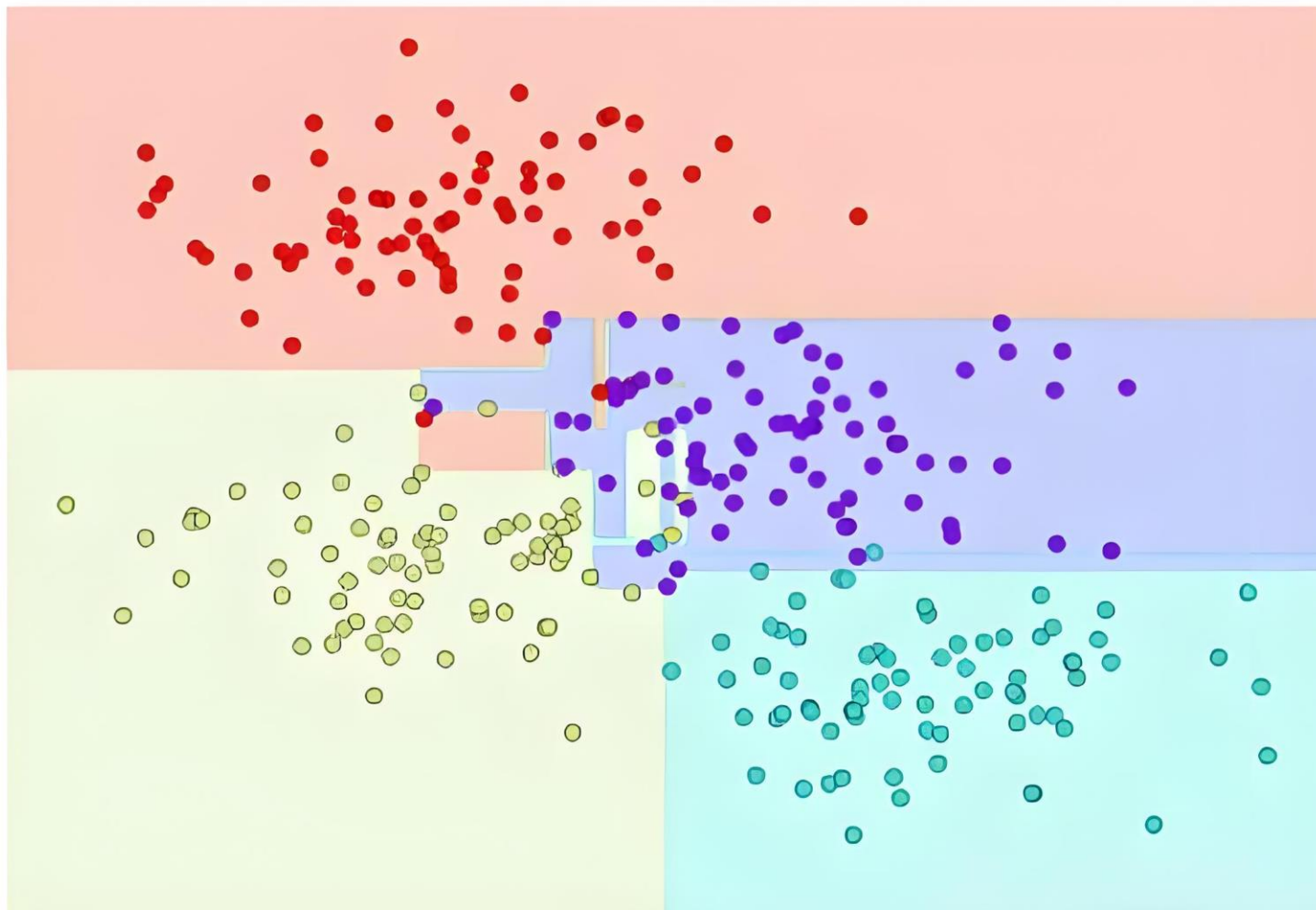
```
def visualize_classifier(model, X, y, ax=None, cmap='rainbow'):
    ax = ax or plt.gca()

    # Plot the training points
    ax.scatter(X[:, 0], X[:, 1], c=y, s=30, cmap=cmap,
               clim=(y.min(), y.max()), zorder=3)
    ax.axis('tight')
    ax.axis('off')
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

    # fit the estimator
    model.fit(X, y)
    xx, yy = np.meshgrid(np.linspace(*xlim, num=200),
                          np.linspace(*ylim, num=200))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

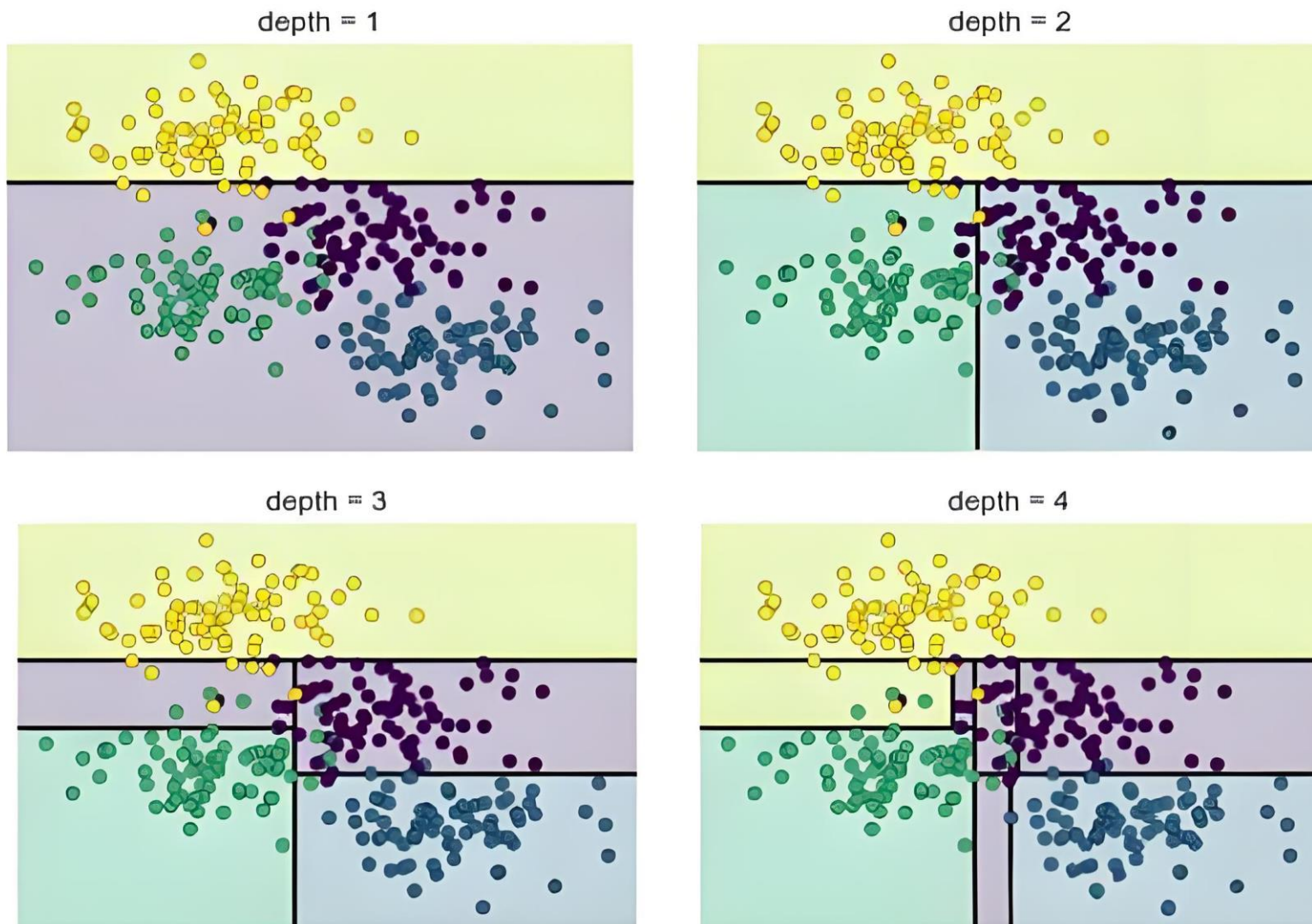
    # Create a color plot with the results
    n_classes = len(np.unique(y))
    contours = ax.contourf(xx, yy, Z, alpha=0.3,
                           levels=np.arange(n_classes + 1) - 0.5,
                           cmap=cmap, clim=(y.min(), y.max()),
                           zorder=1)

    ax.set(xlim=xlim, ylim=ylim)
```

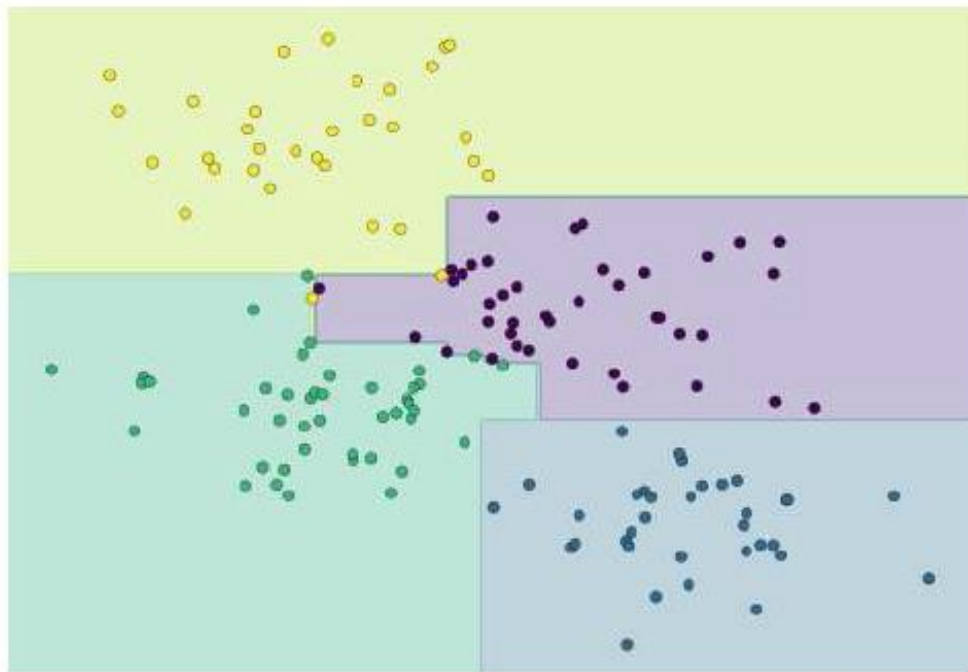
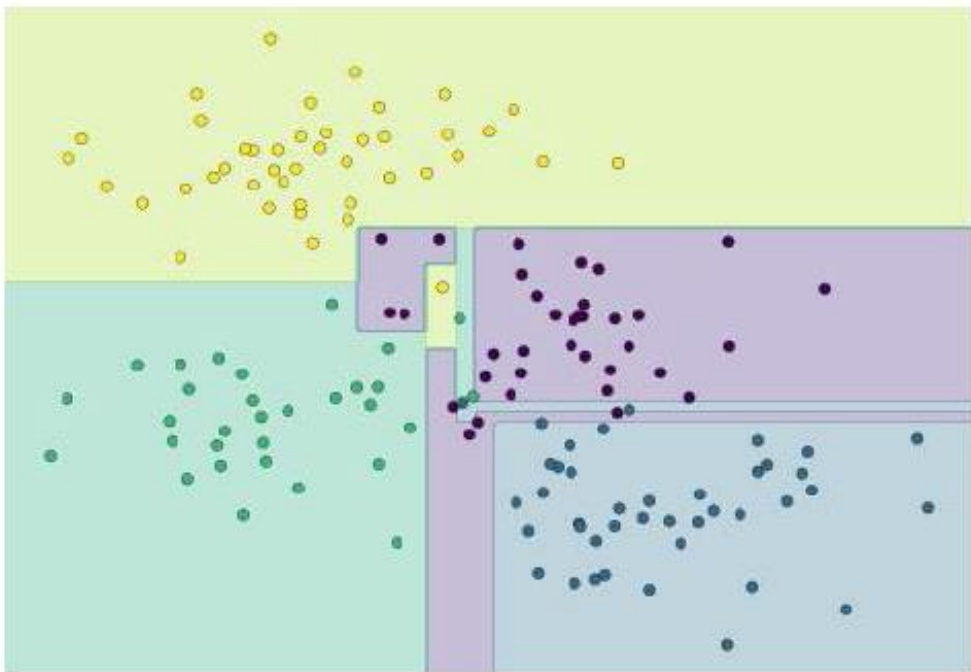



```
from sklearn.tree import DecisionTreeClassifier
tree = DecisionTreeClassifier().fit(X, y)

visualize_classifier(DecisionTreeClassifier(), X, y)
```



四个决策树分类器，使用四种树深度学习同一训练集



两个决策树分类器，分别用同一训练集的一半数据进行学习

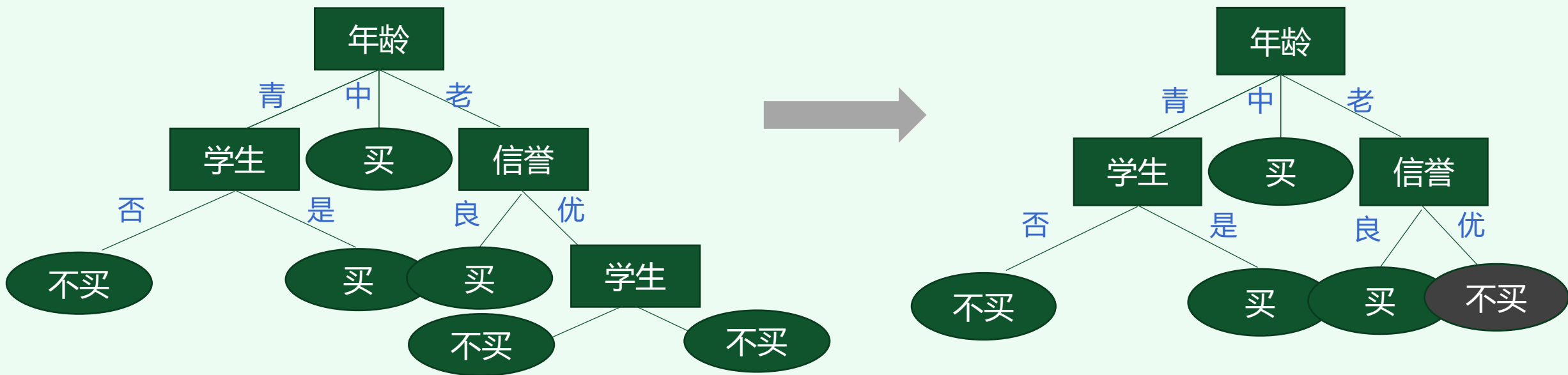


武汉大学
WUHAN UNIVERSITY

03剪枝处理

Decision tree pruning

- 递归生成的决策树往往对训练数据的分类很准确，但对未知的测试数据的分类却没有那么准确，即出现**过拟合**现象。过拟合的原因在于构建了过于复杂的决策树
- 对决策树进行正则化的常用手段是 “**剪枝 (pruning)**”





1 前剪枝和后剪枝

- 剪枝从已生成的树上裁掉一些子树或叶结点，并将其根结点或父结点作为新的叶结点，从而简化分类树模型
- 在树的生成过程中，对结点在划分前进行评估，若不能带来泛化性能提升，则停止划分，将当前结点标记为叶子结点，称为**前（预）剪枝**（prepruning）；在树生成好后进行评估，对树进行回溯，若将该结点对应的子树替换为叶子结点能带来泛化性能的提升，则替换，称为**后剪枝**（postpruning）
- 决策树的生成使用贪心策略（greedy algorithm），每一步只考虑局部最优；剪枝则在全局样本上进行优化
- **实际工程中后剪枝的效果更好，但训练时间开销大**

2 依分类正确率剪枝

- 判定是否剪枝的依据可以是划分前后的**分类正确率**
- 划分前父结点对应的样本**取其中样本最多的类作为分类结果**，会得到一个分类正确率 (Accuracy)。经过属性 a 划分后，统计各分枝上分类正确率。如果正确率有提升，则保留这个划分，否则中止划分（先剪枝）或剪掉该分枝（后剪枝）

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

正确率Accuracy

分类正确的样本数占样本总数的比例 $(TP+TN) / (P+N)$

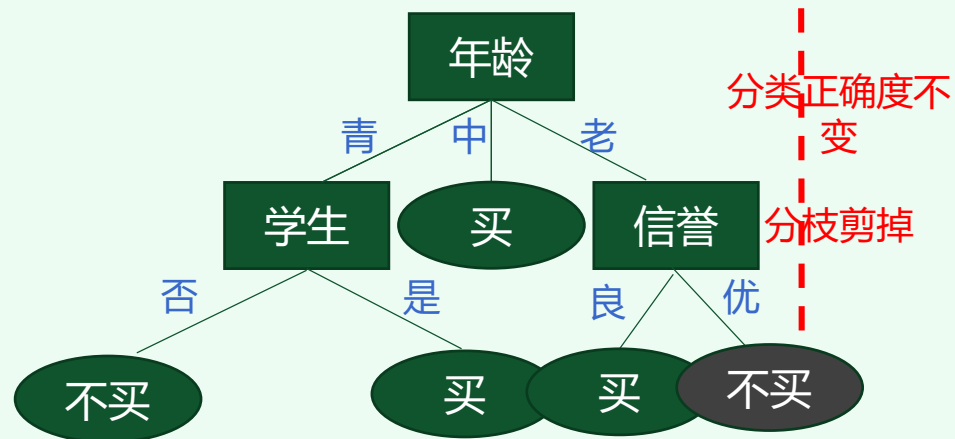
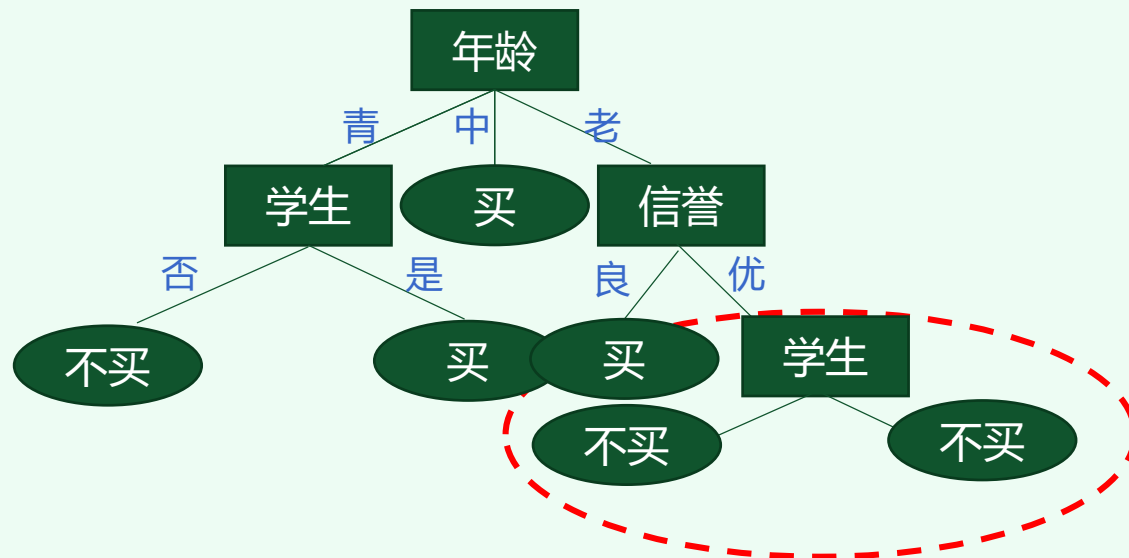
错误率Error

分类错误的样本数占样本总数的比例 $(FP+FN) / (P+N)$

$$\text{Accuracy} = 1 - \text{error}$$



 依分类正确率进行剪枝的例子



$D_{\text{年龄3-信誉2-学生1}}$

计数	年龄	收入	学生	信誉	归类: 买计算机?
64	老	低	是	优	不买

$D_{\text{年龄3-信誉2-学生2}}$

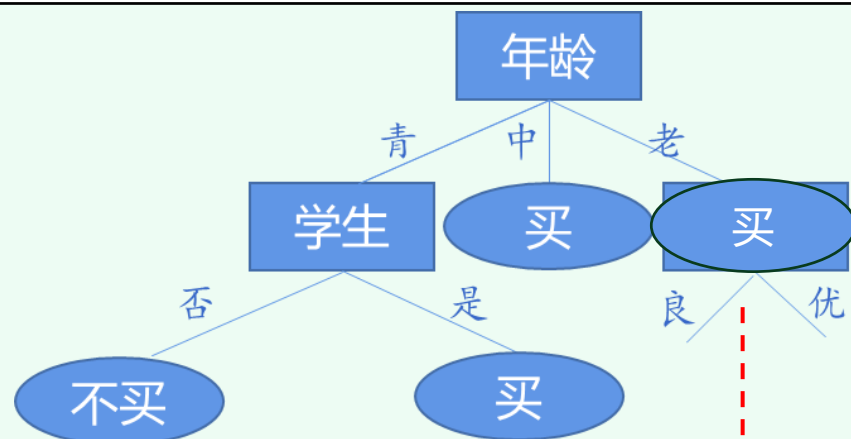
计数	年龄	收入	学生	信誉	归类: 买计算机?
63	老	中	否	优	不买
1	老	中	否	优	买

正确率: $127/128=0.99$

$D_{\text{年龄3-信誉2}}$

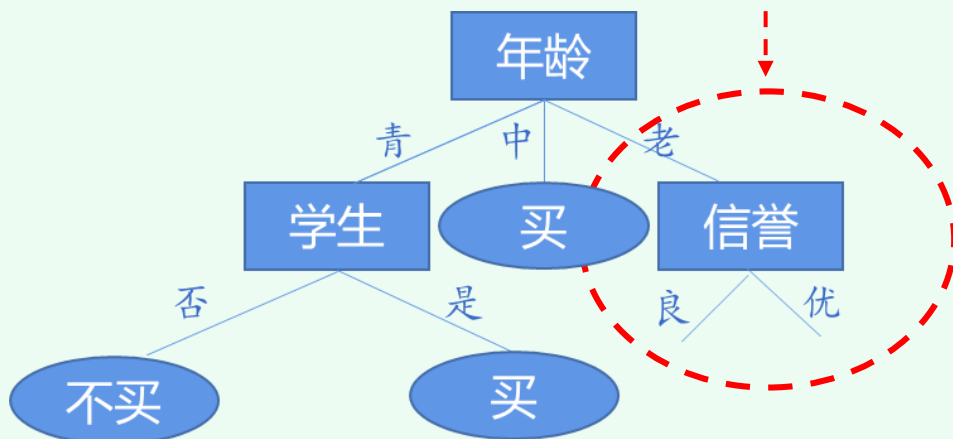
计数	年龄	收入	学生	信誉	归类: 买计算机?
64	老	低	是	优	不买
63	老	中	否	优	不买
1	老	中	否	优	买

正确率: $127/128=0.99$



分类正确度提升

分枝保留


 $D_{\text{年龄3}}$

计数	年龄	收入	学生	信誉	归类: 买计算机?
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
132	老	中	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买

 正确率: $257/384=0.67$
 $D_{\text{年龄3-信誉1}}$

计数	年龄	收入	学生	信誉	归类: 买计算机?
60	老	中	否	良	买
64	老	低	是	良	买
132	老	中	是	良	买

 $D_{\text{年龄3-信誉2}}$

计数	年龄	收入	学生	信誉	归类: 买计算机?
64	老	低	是	优	不买
63	老	中	否	优	不买
1	老	中	否	优	买

 正确率: $383/384=0.997$

3 设计代价函数评估*

- 设树 T 的**叶结点**个数为 $|T|$ ， t 是叶结点对应有 N_t 个样本，其中第 k 类的样本点有 N_{tk} 个， $k = 1, 2, \dots, K$ 。定义 $\text{Ent}(t)$ 为叶结点 t 上的经验熵， $\alpha \geq 0$ 为超参数，则决策树模型的**代价函数**可以定义为：

$$C_\alpha(T) = \sum_{t=1}^{|T|} N_t \text{Ent}(t) + \alpha |T| = - \sum_{t=1}^{|T|} \sum_{k=1}^K N_{tk} \log_2 \frac{N_{tk}}{N_t} + \alpha |T|$$

- 其中叶子结点经验熵：

$$\text{Ent}(t) = - \sum_k \frac{N_{tk}}{N_t} \log_2 \frac{N_{tk}}{N_t}$$

叶结点经验熵越小，说明越
纯，分类正确率越高

对叶结点数量定义规则，是
一种决策树的正则化



- $|T|$ 表示模型**复杂度**。超参数 $\alpha \geq 0$ 控制两者之间的影响。较大的 α 促使选择较简单的模型（树），较小的 α 促使选择较复杂的模型（树）。 $\alpha=0$ 意味着只考虑模型与训练数据的拟合程度，不考虑模型的复杂度
- 决策树训练过程只考虑了通过提高信息增益（或信息增益率）对训练数据进行更好的拟合。而决策树正则化可以减小模型复杂度，提高决策树的**泛化能力**

4 决策树的常见超参数

splitter best
or random

在所有属性中找最好的切分点还是仅在部分属性中

max_depth

树的最大深度

min_samples split

如果某决策结点的样本数少于该值，则不再继续划分

min_samples_leaf

如果叶子结点样本数少于该值，则会被剪枝

max_leaf_nodes

最大叶子结点数，防止过拟合

min_impurity

如果某结点的纯度小于这个阈值则不再划分



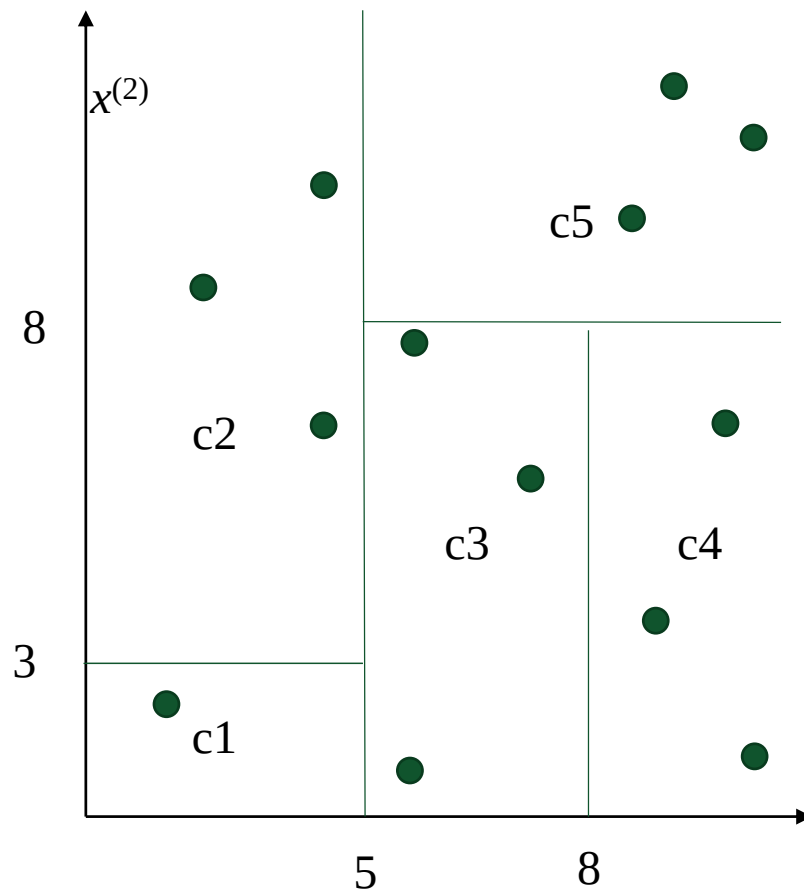
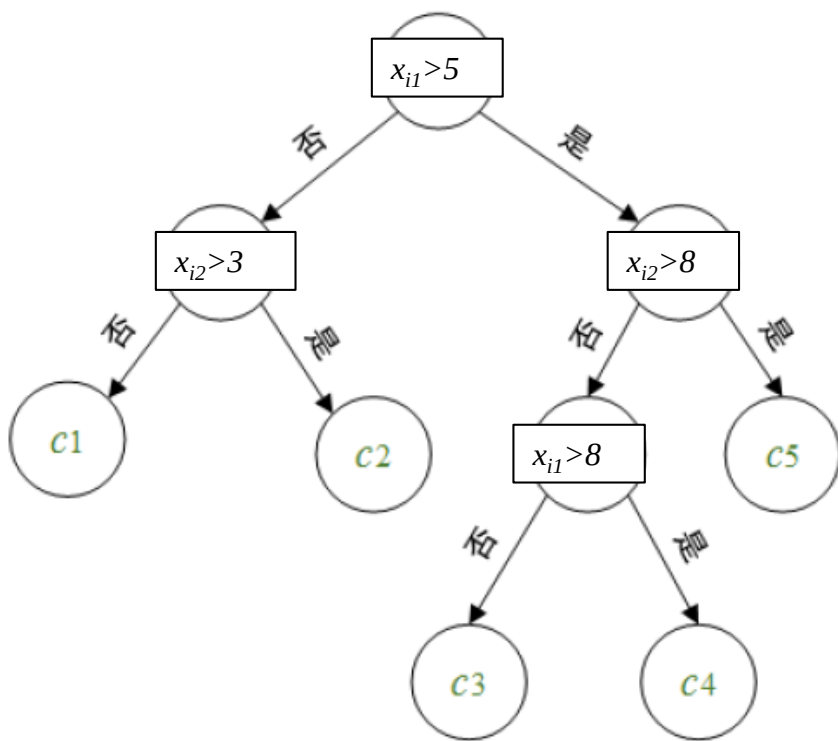
武汉大学
WUHAN UNIVERSITY

04用于回归的决策树

Decision tree is used for regression problems

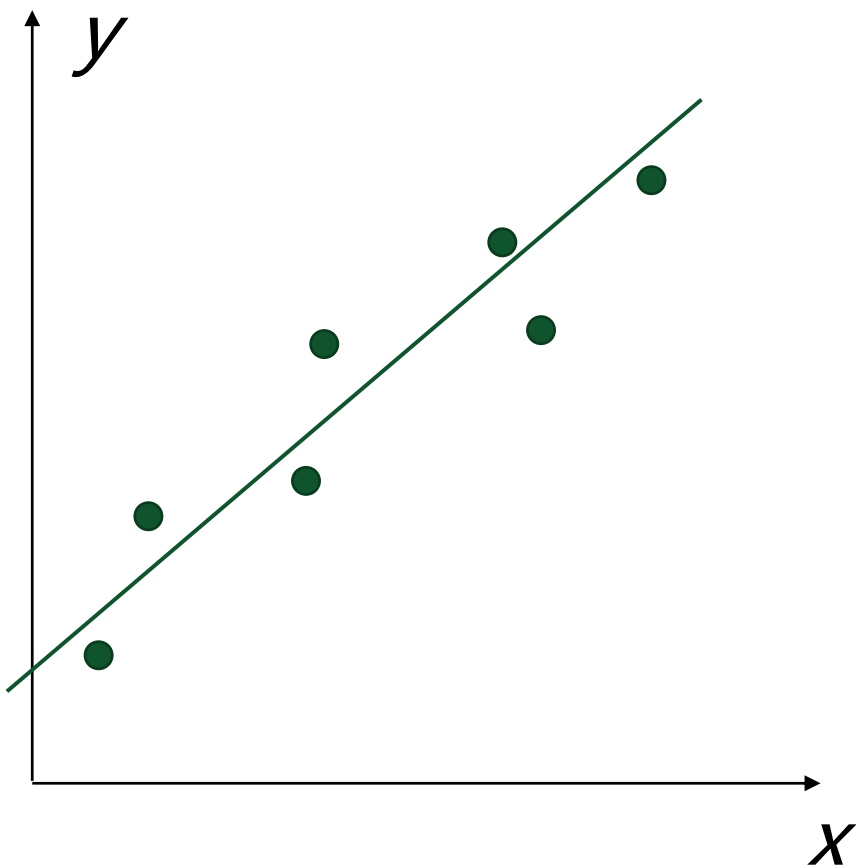
1 决策树假设

- **单变量**决策树的分类边界由一系列**轴平行**的线段组成，将特征空间划分为互不相交的单元(cell)或区域(region)

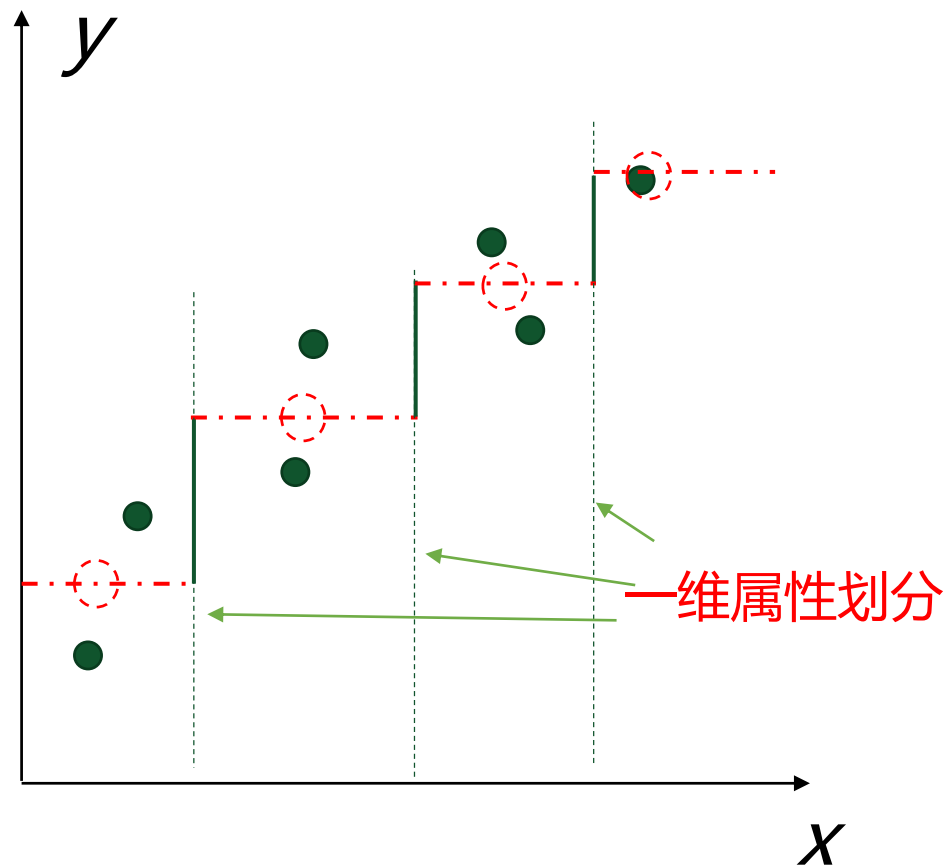


提问：决策树假设的VC维？

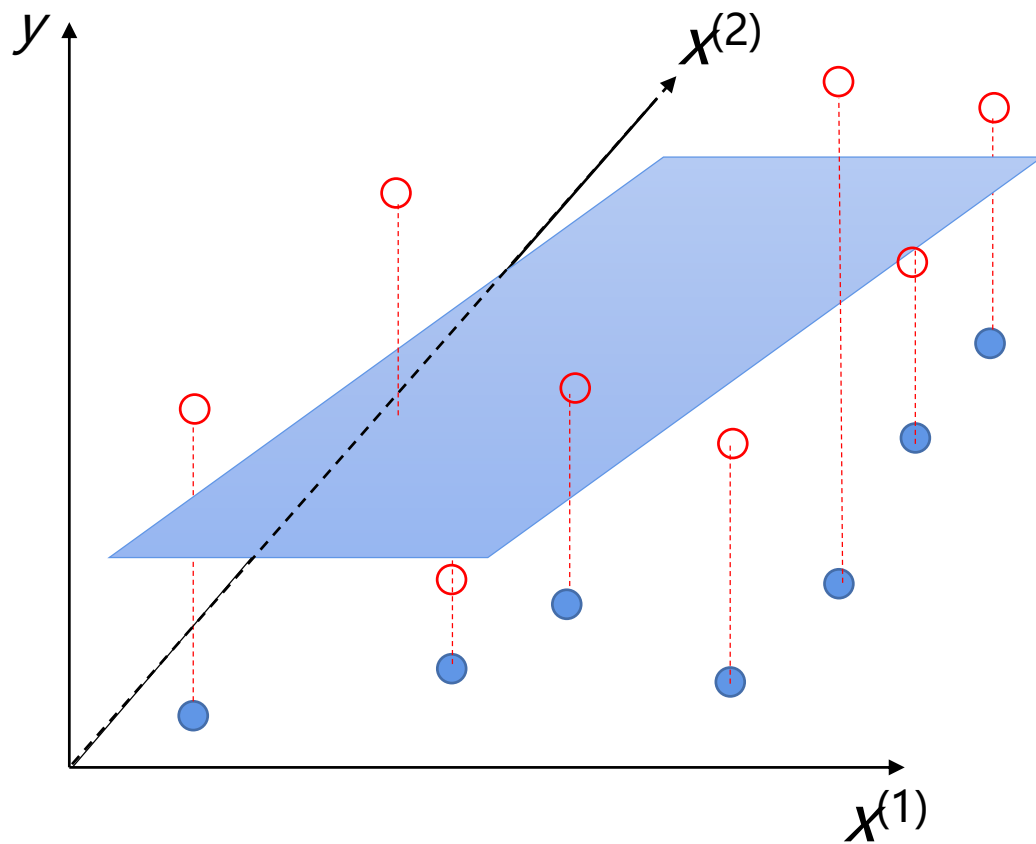
- 联系到回归问题，决策树的回归函数是一组“分段函数”



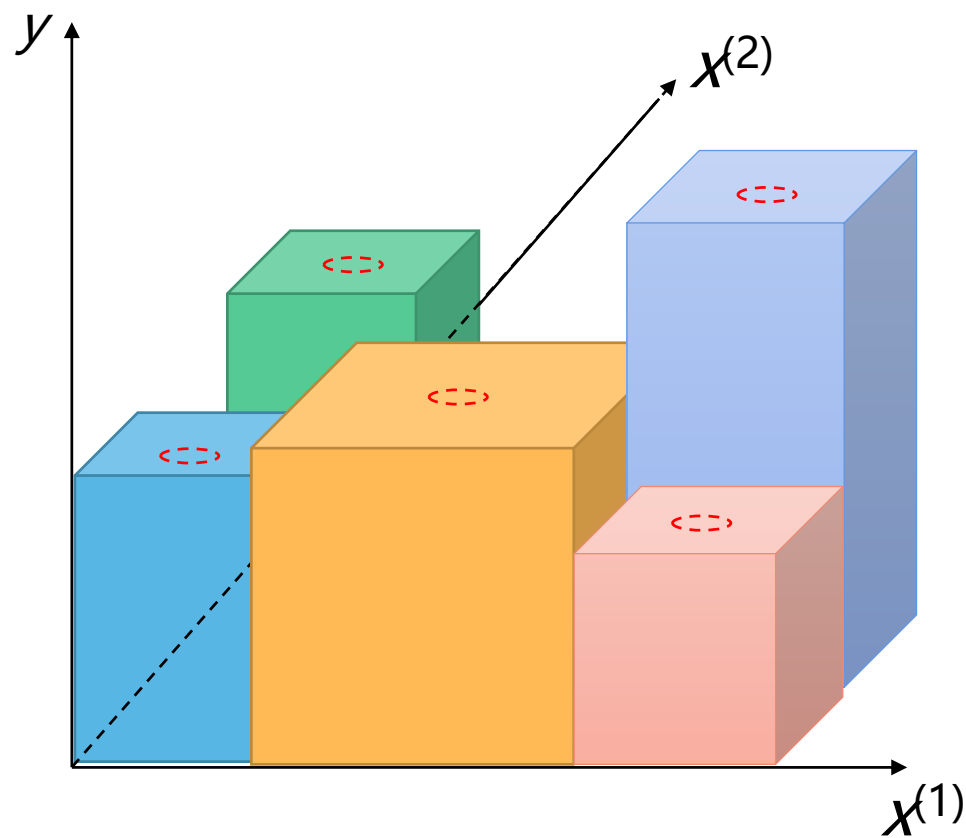
一维样本的线性回归



一维样本的决策树回归（非线性）



二维样本的线性回归



二维样本的决策树回归（非线性）



2 最小二乘回归树

- CART模型可以处理回归任务，其构建的回归决策树被称为“**最小二乘回归树**”。其思想是：对特征空间的划分采用启发式方法，每次划分时逐一考察当前集合中所有特征的所有取值，根据**平方误差最小化准则**选择最优划分点
- 给定数据集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$ ，样本空间由 d 维向量张成， $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{id})^T$, $y_i \in \mathbb{R}$



➤ 将样本第 a 维特征 $x^{(a)}$ 和它的取值 s ，作为划分特征和划分属性点，可定义出两个区域

$R_1(a, s) = \{x | x^{(a)} \leq s\}$ 和 $R_2(a, s) = \{x | x^{(a)} > s\}$ ，最优划分应使得两个区域平方误差和最小

$$\mathbf{a}^*, \mathbf{s}^* = \underset{a, s}{\operatorname{argmin}} [\sum_{x_i \in R_1(a, s)} (y_i - \hat{c}_1)^2 + \sum_{x_i \in R_2(a, s)} (y_i - \hat{c}_2)^2] = \underset{a, s}{\operatorname{argmin}} L(s)$$

- 其中 $\hat{c}_1 = \frac{1}{N_1} \sum_{x_i \in R_1(a, s)} y_i$, $\hat{c}_2 = \frac{1}{N_2} \sum_{x_i \in R_2(a, s)} y_i$

分别为 R_1 区域和 R_2 区域样本的回归预测值

- $\hat{c}_1$ 和 $\hat{c}_2$ 就是各自对应区域内样本标记 y_i 的均值，以此为回归预测值可使得各自区域内平方误差最小，是该区域的最优输出值（预测值）

真实值

预测值

该区域的样本总数



一维决策树回归

x	1	2	3	4	5	6	7	8	9	10
y	5.56	5.7	5.91	6.4	6.8	7.05	8.9	8.7	9	9.05

- 一维数据回归，最优划分属性就是 x 。计算9个划分点的代价函数：

s	1.5	2.5	3.5	4.5	5.5	6.5	7.5	8.5	9.5
c_1	5.56	5.63	5.72	5.89	6.07	6.24	6.62	6.88	7.11
c_2	7.5	7.73	7.99	8.25	8.54	8.91	8.92	9.03	9.05
L(s)	15.72	12.07	8.36	5.78	3.91	1.93	8.01	11.73	15.74

- 划分区域为： $R1=\{1,2,3,4,5,6\}, R2=\{7,8,9,10\}$

- 对应输出值： $\hat{c}_1 = 6.24, \hat{c}_2 = 8.91$

代价函数最小，最优划分

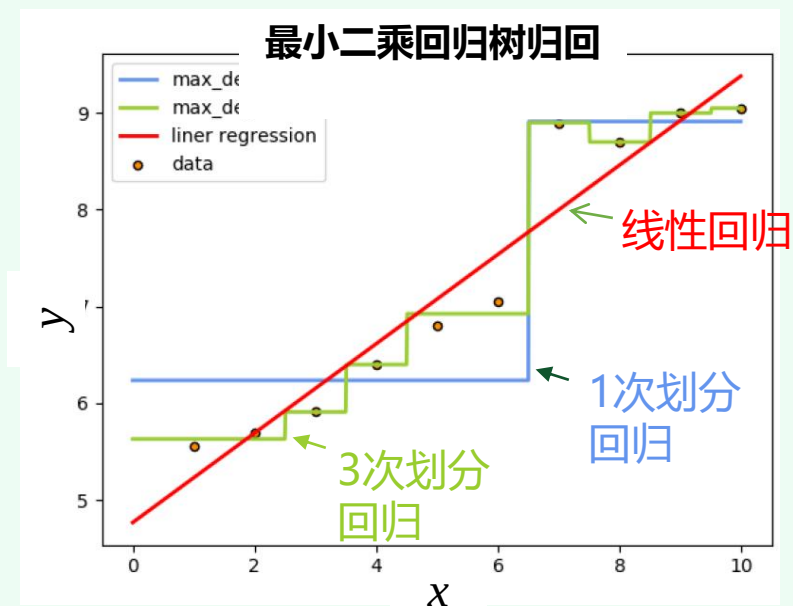
- 下面继续对 $R1$ 区域划分:

s	1.5	2.5	3.5	4.5	5.5
c_1	5.56	5.63	5.72	5.89	6.07
c_2	6.37	6.54	6.75	6.93	7.05

s	1.5	2.5	3.5	4.5	5.5
L(s)	1.3087	0.754	0.2771	0.4368	1.0644

- 对应输出值: $\hat{c}_1 = 5.72$, $\hat{c}_2 = 6.75$

- 得到的两次划分回归树: $T = \begin{cases} 5.72, & x \leq 3.5 \\ 6.75, & 3.5 < x \leq 6.5 \\ 8.91, & x > 6.5 \end{cases}$

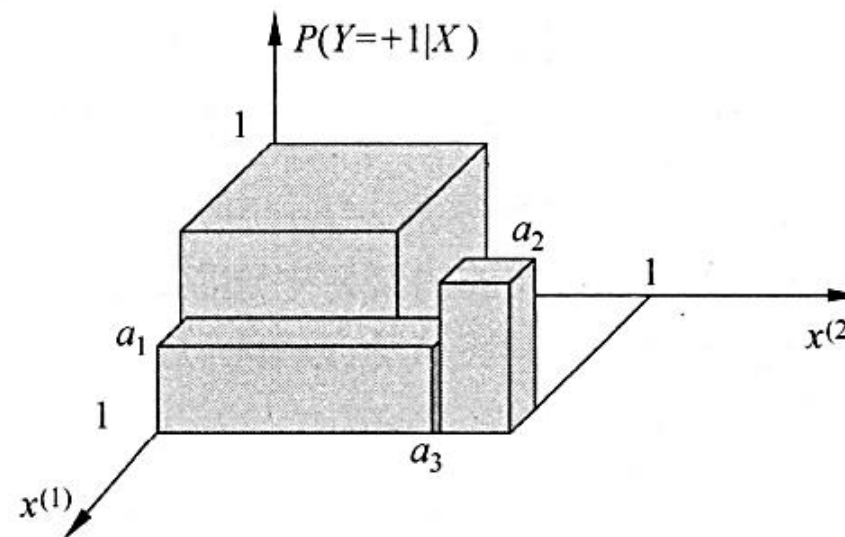




- 决策树学习本质上是从训练数据集中归纳出一组分类规则。与训练数据集不相矛盾的决策树（即能对训练数据进行正确分类的决策树）可能有多个，也可能一个也没有。我们需要的是一个**与训练数据矛盾较小的决策树，同时具有很好的泛化能力**
- 决策树模型易于理解，且对缺失值不敏感，模型训练和执行效率高。但由于采用贪心策略构造，有**可能得不到全局最优树**。同时**模型复杂度（VC维）高，容易过拟合**

1 决策树的概率视角

- 决策树学习是由训练数据集直接估计后验概率。
基于特征空间划分的概率模型有多个。我们选择的后验条件概率模型不仅对训练数据有很好的拟合，而且对未知数据有很好的预测



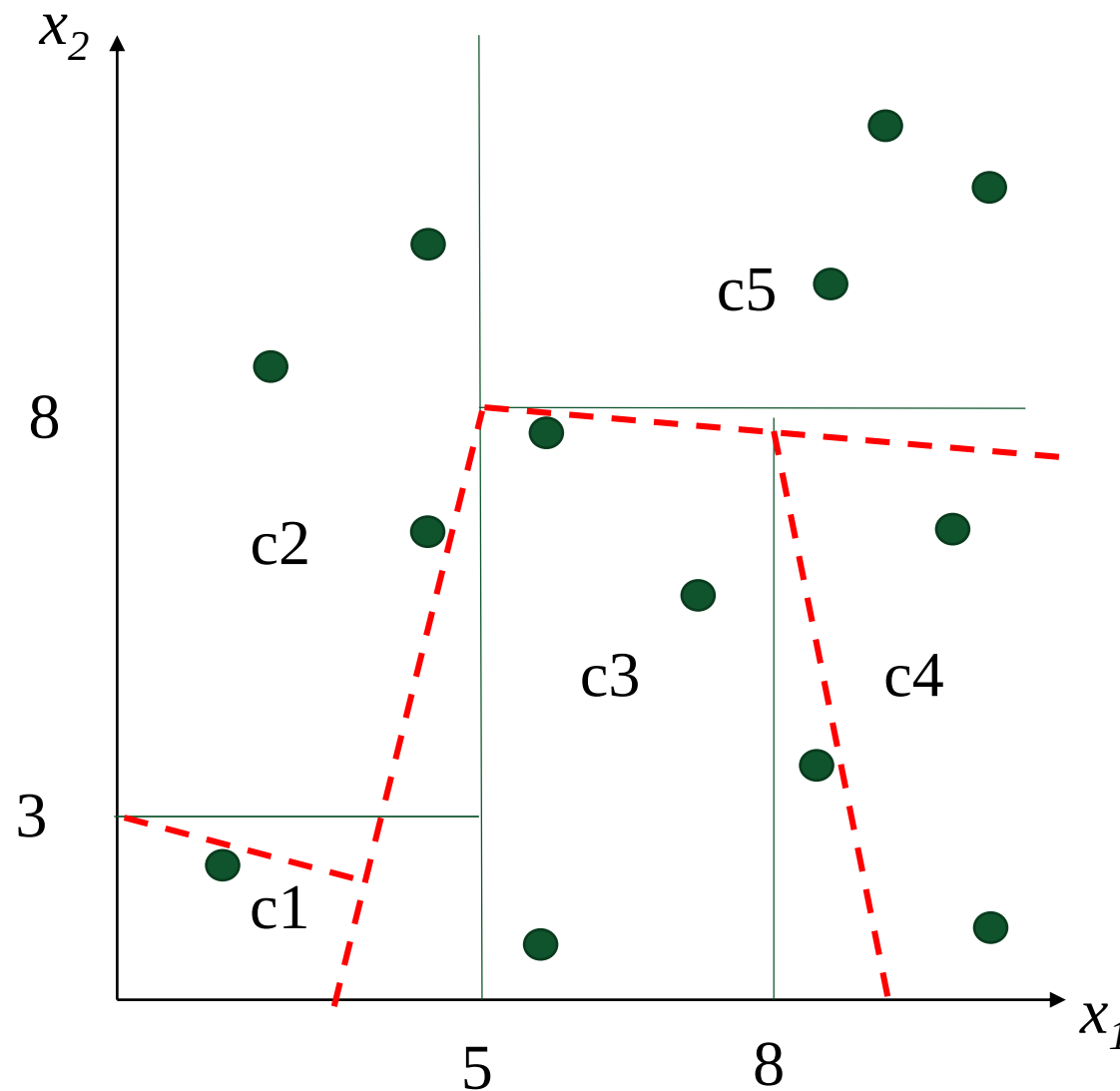
分类后验概率分布

- 决策树的一条路径对应于划分中的一个**单元**。决策树所表示的条件概率分布由各个单元给定样本下条件概率组成
- 各叶结点 (单元) 上的条件概率往往偏向某一个类，即属于某一类的概率较大。决策分类时将该结点的实例强行分到**条件概率大**的那一类中

建议：把决策树分类与朴素贝叶斯分类联系起来学习

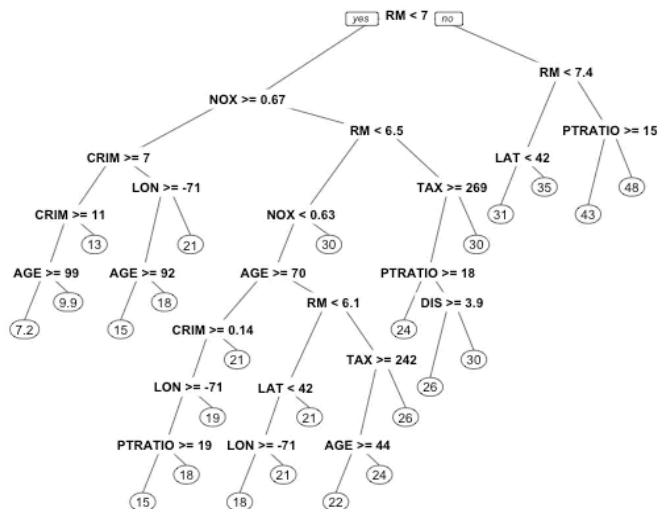
2 多变量决策树

- 将“轴平行的线段”改变为“斜线段”，可以构造更加复杂的决策树。这样的决策树的内部结点不再仅根据某单个属性进行测试，而是对**属性的线性组合**进行测试，即**每个内部结点变为了一个 $\sum w_i a_i = t$ 的线性分类器**，这种决策树称为多变量树（multivariate tree）（见西瓜书p88-91）





模型



- 核心问题：
最优属性划分

策略 (准则)

- 用于分类 (样本不纯度度量) :

$$a^* = \underset{a}{\operatorname{argmin}} \text{Gini_index}(D, a)$$

- 用于回归:

$$a^*, s^* = \underset{a, s}{\operatorname{argmin}} \left[\sum_{x_i \in R_1(a, s)} (y_i - \hat{c}_1)^2 + \sum_{x_i \in R_2(a, s)} (y_i - \hat{c}_2)^2 \right]$$

算法

- 贪心法 (greedy algorithm) :
深度优先生成决策树

* (以CART树为例, ID3和C4.5请自行总结)



武漢大學
WUHAN UNIVERSITY

End of This Part
